



WebLearn

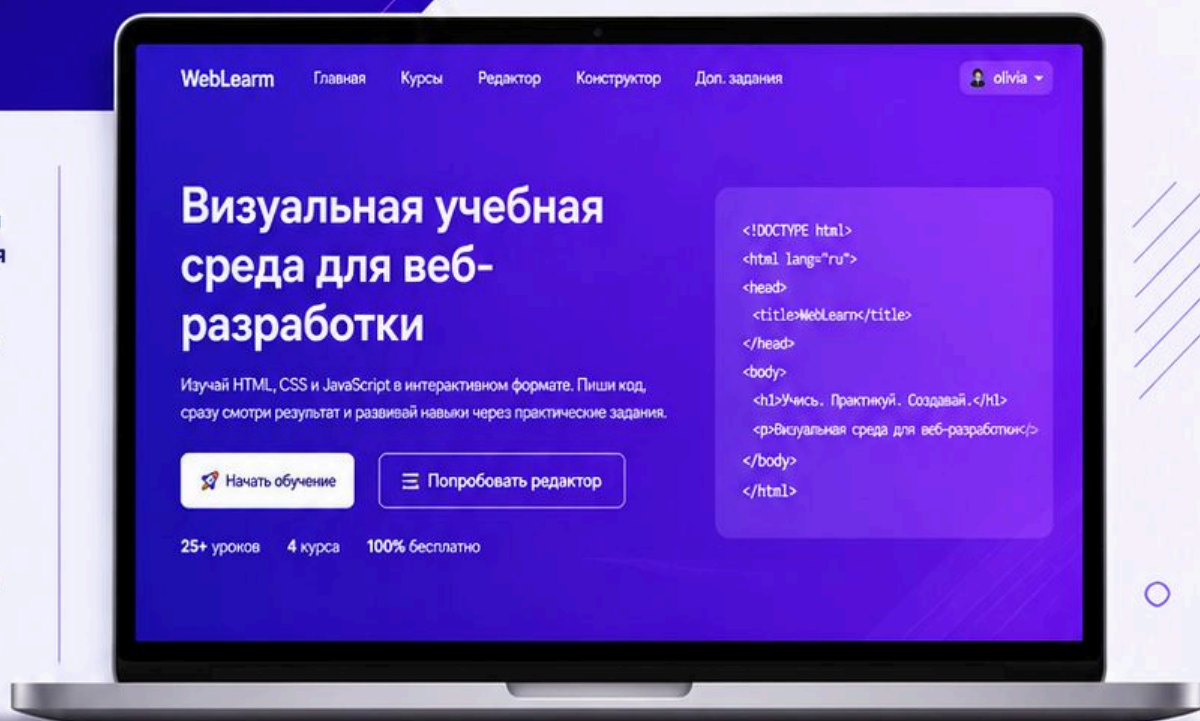
Учись. Практикуй. Создавай.

Методические рекомендации

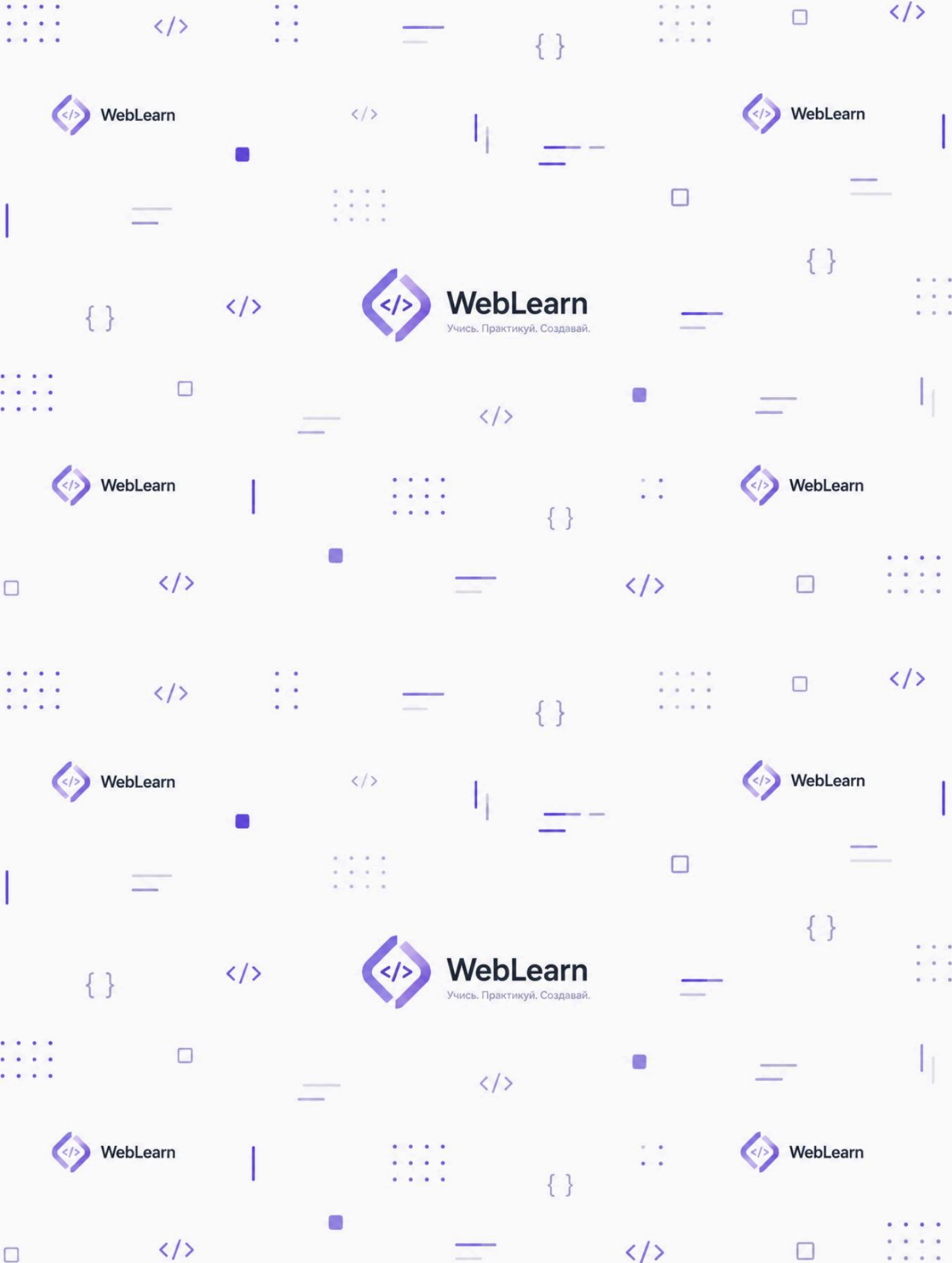
по работе с визуальной
средой WebLearn

Практическое руководство
для преподавателей
и обучающихся

-  Интерактивная среда обучения
-  Теоретические материалы
-  Практические задания
-  Отслеживание прогресса



2026



Методические рекомендации по работе с визуальной учебной средой WebLearn разработаны с целью оказания практической помощи преподавателям и обучающимся при изучении основ веб-разработки с использованием современных цифровых образовательных технологий.

В пособии представлено описание возможностей среды WebLearn, предназначенной для интерактивного обучения HTML, CSS и JavaScript. Среда предоставляет визуальное пространство, в котором обучающиеся могут одновременно писать программный код и наблюдать результат его выполнения в режиме реального времени. Такой подход способствует более эффективному усвоению учебного материала, развитию практических навыков и повышению мотивации к обучению.

Методические рекомендации содержат описание интерфейса среды, инструкции по использованию встроенного редактора кода, практических заданий, системы автоматической проверки и отслеживания прогресса обучающихся. Материалы пособия ориентированы на применение в условиях очного, дистанционного и смешанного обучения.

Методическое пособие рекомендуется преподавателям школ, колледжей и организаций технического и профессионального образования, а также обучающимся, начинающим изучение веб-разработки.

Организация-разработчик:

Казахский национальный женский педагогический университет, кафедра информатики

Разработчики: Бекежановна Алтыншаш Асылхановна, Мирко Оксана Николаевна, Нурмухамедова Рухсарам Молутжановна.

ОГЛАВЛЕНИЕ

1. Общие сведения о визуальной среде WebLearn
2. Термины и определения
3. Начало работы со средой
 - 3.1. Вход в систему
 - 3.2. Навигация по разделам
4. Учебные модули и методика прохождения
 - 4.1. Введение в веб-разработку
 - 4.2. Структура HTML-документа
 - 4.3. Основные теги и атрибуты
 - 4.4. Работа с текстом в HTML
 - 4.5. Списки в HTML
 - 4.6. Гиперссылки
 - 4.7. Изображения
 - 4.8. Таблицы
 - 4.9. Формы
 - 4.10. Семантические элементы HTML5
 - 4.11. Встраивание мультимедиа
 - 4.12. Классы и идентификаторы
 - 4.13. Селекторы, классы и идентификаторы CSS
 - 4.14. Подключение CSS к HTML
 - 4.15. Цвета, текст и шрифты
 - 4.16. Отступы, размеры и блочная модель
 - 4.17. Position и Display
 - 4.18. Flexbox и CSS Grid
 - 4.19. Введение в JavaScript
 - 4.20. Подключение JavaScript к HTML
 - 4.21. Переменные и типы данных
 - 4.22. Операторы и выражения
 - 4.23. Обработка событий
 - 4.24. Циклы
 - 4.25. Функции
 - 4.26. Условные конструкции
 - 4.27. Работа с DOM
 - 4.28. Валидация форм с помощью JavaScript
 - 4.29. Итоговый проект: интерактивный веб-сайт
5. Практические сценарии работы
 - 5.1. Создание личной страницы
 - 5.2. Создание интерактивной формы
 - 5.3. Создание галереи изображений
 - 5.4. Создание калькулятора
 - 5.5. Создание игры «Угадай число»
 - 5.6. Создание интернет-магазина
 - 5.7. Создание мини-социальной сети
 - 5.8. Создание To-Do List
 - 5.9. Создание онлайн-теста
6. Система контроля и оценивания
7. Типичные ошибки и способы их устранения
8. Организация самостоятельной работы обучающихся
9. Заключение

1. Общие сведения о визуальной среде WebLearn

WebLearn — это интерактивная учебная среда, предназначенная для изучения основ веб-программирования в единой цифровой среде. Среда сочетает теоретическое обучение, практическую работу с кодом и визуальное проектирование интерфейсов, что позволяет пользователям одновременно изучать материал и сразу применять полученные знания на практике. Основная особенность среды заключается в том, что пользователь может писать HTML-, CSS- и JavaScript-код и мгновенно видеть результат своей работы в окне предварительного просмотра без необходимости использовать дополнительные программы или сложные среды разработки.

Основные возможности среды:

- **Единая среда для обучения и практики** — все необходимые инструменты размещены внутри одной среды. Пользователю не требуется устанавливать сторонние редакторы кода или дополнительные программы. Теория, практика, визуальный конструктор и система проверки заданий доступны в одном рабочем пространстве.
- **Поэтапное освоение HTML, CSS и JavaScript** — учебный материал организован по принципу постепенного усложнения. Пользователь последовательно изучает основы структуры веб-страниц, стилизации интерфейсов и создания интерактивных элементов, закрепляя знания с помощью практических заданий.
- **Доступ к заданиям разного уровня сложности** — среда содержит как базовые упражнения для начинающих, так и более сложные практические проекты. Это позволяет учитывать уровень подготовки обучающихся и постепенно повышать сложность выполняемых работ.
- **Возможность отправки решения на проверку** — выполненные задания могут быть сохранены и отправлены преподавателю или наставнику для проверки. Пользователь получает комментарии, рекомендации и обратную связь, что помогает выявлять ошибки и улучшать качество работы.
- **Визуальная поддержка в виде конструктора страниц** — встроенный конструктор помогает создавать веб-страницы с использованием готовых компонентов интерфейса. Такой формат особенно полезен для начинающих пользователей, поскольку упрощает понимание структуры веб-документов и снижает количество технических ошибок при обучении.

Использование среды WebLearn способствует развитию практических навыков программирования, логического мышления, самостоятельности и творческого подхода к созданию веб-проектов. Благодаря сочетанию теории, практики и визуального проектирования процесс обучения становится более интерактивным, понятным и эффективным.

2. Термины и определения

WebLearn — веб-среда для обучения основам веб-программирования, предназначенная для изучения HTML, CSS и JavaScript в интерактивной форме. Среда объединяет теоретические материалы, практические задания, встроенный редактор кода и визуальный конструктор интерфейсов в единой учебной среде. WebLearn позволяет пользователям сразу видеть результат выполнения заданий, получать обратную связь и постепенно развивать навыки веб-разработки.

Курс — структурированный набор учебных модулей, объединённых общей тематикой и направленных на последовательное освоение определённого раздела веб-программирования. Курс включает теоретические материалы, примеры, практические задания, тестирование и итоговые проекты.

Урок — отдельная единица обучения внутри курса, содержащая теоретический материал, демонстрационные примеры, практические задания и рекомендации по выполнению работы. Каждый урок ориентирован на формирование конкретного навыка или изучение отдельной темы.

Редактор — встроенная рабочая область платформы, предназначенная для ввода, редактирования и запуска HTML-, CSS- и JavaScript-кода. Редактор позволяет пользователю выполнять практические задания, изменять код и сразу просматривать результат выполненных действий.

Превью — область мгновенного отображения результата работы кода. После внесения изменений пользователь может сразу увидеть, как будет выглядеть веб-страница или как работают интерактивные элементы, что упрощает процесс обучения и поиска ошибок.

Конструктор — визуальный инструмент платформы для создания веб-страниц с помощью готовых блоков и элементов интерфейса без необходимости полного ручного написания кода. Конструктор помогает пользователям изучать структуру веб-страниц и автоматически формирует соответствующий HTML- и CSS-код.

Дополнительное задание — практическая работа повышенного или дополнительного уровня сложности, выполняемая вне основной последовательности уроков. Такие задания предназначены для закрепления изученного материала, развития самостоятельности и расширения практических навыков обучающихся.

Обратная связь — система комментариев и рекомендаций преподавателя или наставника по результатам выполненной работы. Обратная связь помогает пользователю выявлять ошибки, улучшать качество кода, понимать причины допущенных неточностей и корректировать дальнейшую учебную деятельность.

Модуль — крупный структурный блок курса, объединяющий несколько уроков, посвящённых одной теме или направлению (например, HTML-разметка, стилизация CSS или основы JavaScript). Модули позволяют выстраивать обучение по принципу постепенного усложнения материала.

Практика — раздел урока или курса, включающий задания для самостоятельного выполнения. Практика направлена на закрепление

теоретических знаний и развитие навыков программирования через выполнение реальных задач.

Тестирование — форма контроля знаний, включающая вопросы с выбором ответа, вводом кода или короткими заданиями. Используется для проверки усвоения материала после прохождения темы или модуля.

Проект — итоговая или промежуточная работа, в рамках которой обучающийся создаёт полноценный веб-ресурс или приложение, применяя изученные технологии HTML, CSS и JavaScript.

Шаблон — заранее подготовленная структура веб-страницы или интерфейса, которую пользователь может использовать как основу для выполнения заданий или создания собственных проектов.

Подсказка — вспомогательный элемент интерфейса, предоставляющий пользователю дополнительную информацию, пример кода или рекомендацию по выполнению задания.

Автопроверка — функция платформы, которая автоматически анализирует решение пользователя и сообщает о наличии ошибок или соответствии требованиям задания.

Синтаксический анализ (линтинг) — процесс автоматической проверки кода на наличие ошибок, несоответствий стилю написания и потенциальных проблем.

Сохранение прогресса — функция платформы, позволяющая фиксировать выполненные уроки, задания и результаты тестов для отслеживания учебного прогресса пользователя.

Профиль пользователя — персональная страница обучающегося, содержащая информацию о пройденных курсах, достижениях, статистике и уровне освоения материала.

Достижение — элемент системы мотивации, выдаваемый пользователю за выполнение определённых условий (например, завершение модуля, успешное прохождение теста или выполнение проекта).

Рейтинг — система оценки активности и успеваемости пользователя, основанная на выполненных заданиях, тестах и проектах.

Навигация — система элементов интерфейса, позволяющая пользователю перемещаться между курсами, уроками, заданиями и разделами платформы.

Интерактивный пример — демонстрационный код или элемент, с которым пользователь может взаимодействовать и изменять его параметры для лучшего понимания темы.

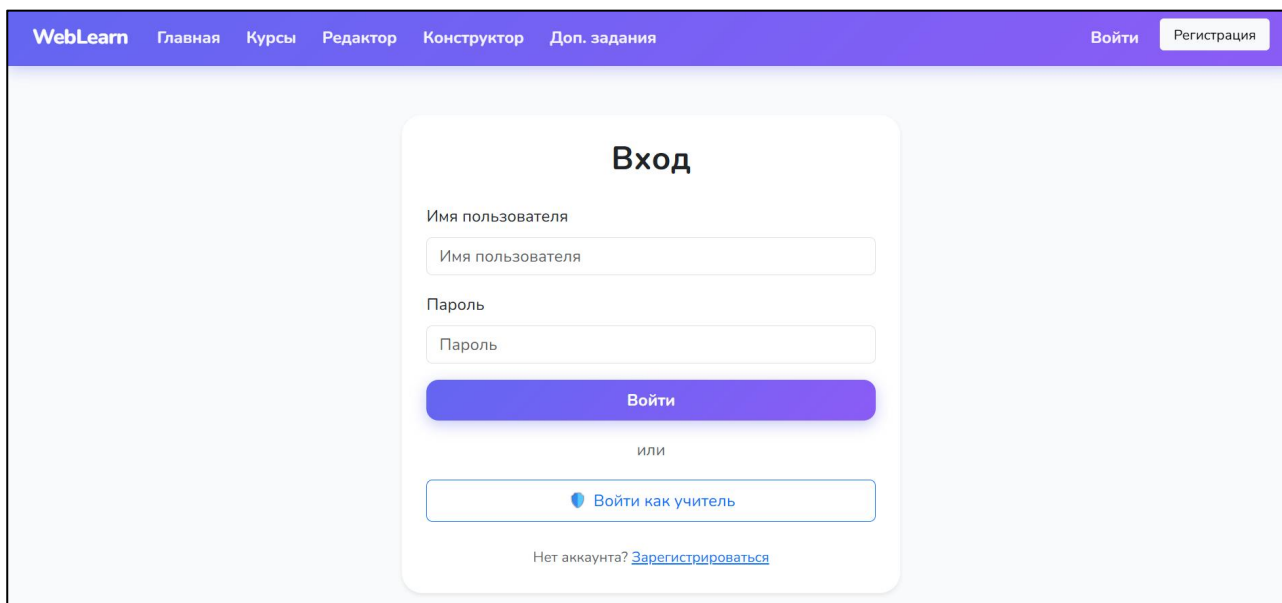
Режим обучения — специальный формат работы платформы, в котором пользователю последовательно предлагаются материалы, задания и проверки без возможности пропуска ключевых тем.

3. Начало работы со средой

3.1. Вход в систему

1. Откройте страницу среды в браузере.
2. Перейдите в раздел входа через верхнее меню.
3. Введите логин и пароль учетной записи.
4. Нажмите кнопку «Войти».
5. Убедитесь, что открылась рабочая область пользователя.

Если войти не получается, проверьте раскладку клавиатуры, Caps Lock, логин и пароль. Также убедитесь, что есть интернет. Если ошибка не исчезает, обратитесь к преподавателю.



3.2. Навигация по разделам

- Раздел «Главная» является стартовой страницей среды WebLearn и предназначен для быстрого доступа к основным функциям системы. В данном разделе пользователь получает общую информацию о среде, текущих курсах, последних действиях и доступных возможностях. На главной странице могут отображаться уведомления, рекомендации по обучению, прогресс прохождения курса и ссылки на наиболее часто используемые разделы. Также здесь размещаются быстрые кнопки перехода к редактору, учебным модулям и практическим заданиям. Раздел помогает пользователю быстро ориентироваться в системе и продолжать обучение без необходимости поиска нужных материалов.
- Раздел «Курсы» представляет собой основную учебную траекторию среды и содержит структурированные программы обучения по веб-программированию. Каждый курс состоит из отдельных модулей и уроков, расположенных в логической последовательности — от базовых тем к более сложным. Внутри курсов пользователь может изучать теоретический материал, выполнять практические задания, проходить тестирование и отслеживать

собственный прогресс. Раздел обеспечивает поэтапное формирование навыков работы с HTML, CSS и JavaScript, а также позволяет преподавателю контролировать выполнение заданий и результаты обучающихся.

- Раздел «Редактор» предназначен для выполнения практических заданий и работы с программным кодом непосредственно в браузере. Пользователь может вводить, изменять и запускать HTML-, CSS- и JavaScript-код в единой рабочей области. После внесения изменений результат автоматически отображается в окне предварительного просмотра, что позволяет сразу видеть эффект выполненных действий. Редактор поддерживает сохранение промежуточных результатов, отправку работы на проверку преподавателю и повторное редактирование проекта. Данный раздел играет ключевую роль в формировании практических навыков веб-разработки.

- Раздел «Конструктор» представляет собой визуальный инструмент для создания веб-страниц с помощью готовых блоков и элементов интерфейса. Пользователь может добавлять текстовые блоки, изображения, кнопки, формы и другие компоненты методом перетаскивания без необходимости полного ручного написания кода. Во время работы конструктор автоматически формирует HTML- и CSS-разметку, что позволяет обучающимся одновременно изучать структуру веб-страниц и принципы построения интерфейсов. Такой подход особенно полезен для начинающих пользователей, поскольку упрощает процесс освоения веб-разработки и снижает количество технических ошибок.

- Раздел «Дополнительные задания» предназначен для расширенной практики и углублённого закрепления изученного материала. В данном разделе размещаются задачи повышенного уровня сложности, мини-проекты, интерактивные упражнения и творческие задания, которые помогают обучающимся применять полученные знания в нестандартных ситуациях. Дополнительные задания способствуют развитию самостоятельности, логического мышления, навыков проектирования интерфейсов и написания программного кода. Выполнение таких работ позволяет пользователям улучшать качество своих проектов, расширять практический опыт и повышать уровень подготовки в области веб-программирования.

Удобный порядок работы: сначала уроки в разделе «Курсы», потом практика в «Редакторе», а затем дополнительные задания.

4. Учебные модули и методика прохождения

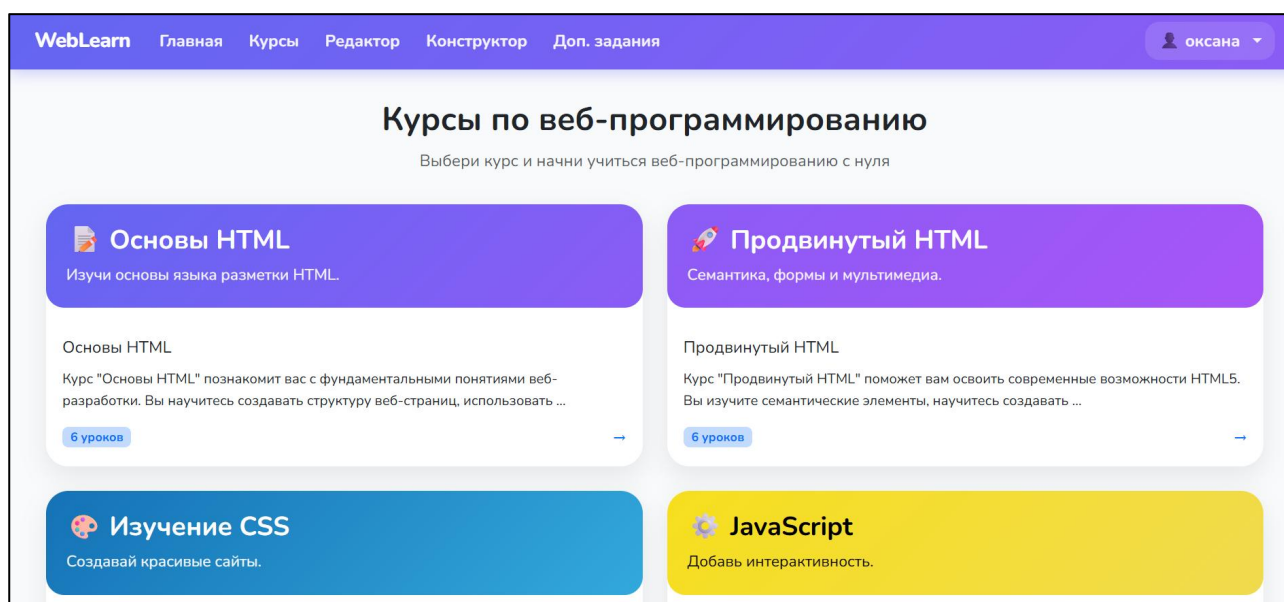
Программа обучения разделена на несколько учебных модулей, каждый из которых посвящён отдельной теме и направлен на формирование конкретных практических навыков. Последовательное изучение модулей позволяет постепенно осваивать материал, закреплять полученные знания и применять их на практике.

Для каждого модуля определены чёткие цели обучения, которые помогают понять, какие компетенции и умения должны быть сформированы по итогам его прохождения. Перед началом работы рекомендуется ознакомиться с целями и ожидаемыми результатами, чтобы лучше ориентироваться в содержании курса.

Каждый модуль включает пошаговую методику выполнения заданий. Учебный материал изучается последовательно: от теоретических основ к практическим упражнениям и самостоятельным заданиям. Такой подход способствует более глубокому пониманию темы и позволяет избежать типичных ошибок при выполнении работы.

Кроме того, в каждом модуле содержатся рекомендации и полезные советы, которые помогут эффективно организовать процесс обучения, правильно выполнять задания и быстрее находить решения возникающих проблем. Для достижения наилучшего результата рекомендуется выполнять все упражнения, внимательно анализировать полученные результаты и регулярно повторять изученный материал.

Последовательное прохождение всех модулей обеспечивает комплексное освоение курса и способствует развитию устойчивых практических навыков, необходимых для дальнейшей профессиональной деятельности.



4.1. Модуль «Введение в веб-разработку»

Цель данного модуля заключается в понимании изучаемой темы и формировании практических навыков её применения. Для достижения наилучшего результата рекомендуется проходить материал последовательно, не пропуская теоретические разделы, примеры и практические задания. Такой подход позволяет лучше усвоить материал и закрепить полученные знания на практике.

Работу над модулем следует начинать с изучения теоретического материала. После ознакомления с теорией необходимо внимательно разобрать демонстрационный пример, обращая внимание на структуру кода и используемые решения. Затем рекомендуется самостоятельно повторить пример в редакторе без копирования, чтобы лучше понять принцип его работы. При выполнении заданий полезно придерживаться следующих рекомендаций:

- перед началом работы сформулировать ожидаемый результат;
- изменять код небольшими шагами и сразу проверять изменения;
- использовать понятные и логичные имена классов и переменных;
- соблюдать аккуратную структуру и читаемость кода;
- после завершения работы проводить самопроверку по чек-листу.

Частыми ошибками являются забытые закрывающие теги, опечатки в CSS-коде, неправильные отступы и нарушение структуры страницы. Поэтому рекомендуется проверять код после каждого шага.

Порядок выполнения работы:

1. Откройте соответствующий урок модуля.
2. Изучите теорию и демонстрационный пример.
3. Повторите пример самостоятельно.
4. Создайте собственный вариант решения.
5. Проверьте корректность отображения результата.
6. Сохраните работу и при необходимости отправьте её на проверку.

The screenshot displays the WebLearn platform interface. At the top, a purple navigation bar contains the 'WebLearn' logo and links to 'Главная', 'Курсы', 'Редактор', 'Конструктор', and 'Доп. задания'. A user profile 'admin' is visible in the top right. Below the navigation bar, a breadcrumb trail reads 'Курсы / Основы HTML / Введение в веб-разработку'. The main content area is divided into two columns. The left column, titled 'Основы HTML', lists several topics: 'Введение в веб-разраб...', 'Структура HTML-докуме...', 'Основные теги и атрибу...', 'Работа с текстом в HTML', 'Списки в HTML', and 'Гиперссылки'. The right column, titled 'Введение в веб-разработку', shows the current lesson. It includes a green 'Выполнено' (Completed) badge. The text explains that web development is the creation of websites and web applications used daily via the internet, with examples like YouTube, Instagram, and Kaspi.kz. A blue box highlights an 'Interesting fact' that modern websites are built using HTML, CSS, and JavaScript. Below this, a section titled 'Что такое сайт?' (What is a website?) defines a website as a set of interconnected web pages accessible via a browser at a specific address. It also lists examples of websites: Google (search), YouTube (video), and Kaspi.kz (online bank).

4.2. Модуль «Структура HTML-документа»

Цель модуля заключается в изучении темы и формировании практических навыков её применения. Для успешного освоения материала рекомендуется проходить уроки последовательно, не пропуская теоретические разделы и практические задания. Такой подход помогает лучше понять принципы работы и закрепить полученные знания.

Начните обучение с внимательного изучения теории. После этого разберите демонстрационный пример, обращая внимание на структуру кода и используемые элементы. Затем самостоятельно повторите пример в редакторе без копирования готового решения. Это позволит лучше запомнить материал и понять логику выполнения задания.

Во время выполнения заданий рекомендуется придерживаться следующих правил:

- перед началом работы определяйте ожидаемый результат;
- вносите изменения в код постепенно и сразу проверяйте результат;
- используйте понятные названия классов и переменных;
- соблюдайте структуру и читаемость кода;
- выполняйте самопроверку после завершения задания.

Частыми ошибками являются пропущенные закрывающие теги, опечатки в CSS-коде, неправильные отступы и нарушение структуры страницы. Для своевременного обнаружения ошибок необходимо проверять код после каждого шага работы.

Порядок выполнения:

1. Откройте урок модуля.
2. Изучите теорию и пример.
3. Повторите пример самостоятельно.
4. Создайте собственное решение.
5. Проверьте результат.
6. Сохраните работу и при необходимости отправьте её на проверку.

The screenshot displays the WebLearn platform interface. At the top, a purple navigation bar contains links: Главная, Курсы, Редактор, Конструктор, and Доп. задания. A user profile icon labeled 'admin' is on the right. Below the navigation bar, a breadcrumb trail reads: Курсы / Основы HTML / Структура HTML-документа. The main content area is divided into two columns. The left column, titled 'Основы HTML', lists several topics: Введение в веб-разрабо..., Структура HTML-докум..., Основные теги и атрибу..., Работа с текстом в HTML, Списки в HTML, and Гиперссылки. The right column, titled 'Структура HTML-документа', features a green 'Выполнено' (Completed) button in the top right corner. The text in this column explains that any web page is created according to specific rules and that an HTML document must have a clear structure. A light blue callout box states: 'Представьте: HTML-документ похож на дом. У дома есть фундамент, стены и комнаты. Точно так же веб-страница состоит из нескольких обязательных частей.' Below this, a section titled 'Основные элементы HTML-документа' lists four elements: <!DOCTYPE html> (notifies the browser of the HTML5 version), <html> (the main container), <head> (contains meta-information), and <body> (contains the visible content). At the bottom of the right column, a section titled 'Общая структура страницы' is partially visible.

4.3. Модуль «Основные теги и атрибуты»

Цель модуля: изучить основные HTML-теги и атрибуты, научиться создавать структуру веб-страницы и правильно оформлять элементы с помощью атрибутов.

Рекомендуется проходить материал последовательно и выполнять все практические задания. Сначала внимательно изучите теоретический материал, затем разберите демонстрационный пример и повторите его самостоятельно без копирования. После этого создайте собственный вариант веб-страницы, используя изученные теги и атрибуты. Такой подход позволит лучше понять принцип работы HTML и закрепить полученные знания на практике. перед началом задания определите, как должна выглядеть веб-страница;

- добавляйте теги и атрибуты постепенно, проверяя результат после каждого шага;
- используйте понятные имена классов и атрибутов;
- соблюдайте правильную структуру HTML-документа;
- после завершения выполните самопроверку по чек-листу.

Частые ошибки: незакрытые теги, неправильное использование атрибутов, ошибки в написании тегов, нарушение вложенности элементов и отсутствие структуры страницы. Проверяйте код после каждого изменения.

1. Откройте урок модуля «Основные теги и атрибуты».
2. Прочитайте теоретический материал и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Добавьте собственные элементы и атрибуты на страницу.
5. Проверьте корректность отображения и структуры страницы.
6. Выполните самопроверку и убедитесь в отсутствии ошибок.
7. Сохраните результат и при необходимости отправьте работу на проверку.

WebLearn Главная Курсы Редактор Конструктор Доп. задания admin

Курсы / Основы HTML / Основные теги и атрибуты

Основы HTML

- Введение в веб-разрабо...
- Структура HTML-докуме...
- Основные теги и атриб...**
- Работа с текстом в HTML
- Списки в HTML
- Гиперссылки

Основные теги и атрибуты Выполнено

HTML состоит из специальных команд — тегов. Именно с их помощью браузер понимает, какие элементы нужно показать на странице: заголовок, текст, изображение, кнопку или ссылку.

Важно: тег можно сравнить с инструкцией для браузера. Он сообщает, как должен выглядеть определённый элемент страницы.

📖 Что такое тег?

Тег — это элемент HTML, который заключается в угловые скобки `< >`.

```
<p>Это параграф</p>
```

В данном примере тег `<p>` создаёт обычный текстовый абзац.

★ Основные HTML-теги

4.4. Модуль «Работа с текстом в HTML»

Цель модуля: научиться работать с текстом в HTML, использовать заголовки, абзацы, списки, выделение текста и другие текстовые элементы для правильного оформления веб-страницы.

Рекомендуется изучать материал последовательно и выполнять практические задания после каждой темы. Сначала прочитайте теорию, затем разберите пример и повторите код вручную. После этого создайте собственный вариант оформления текста.

- перед началом задания определите, как должен выглядеть текст на странице;
- добавляйте текстовые элементы постепенно и сразу проверяйте результат;
- используйте подходящие теги для заголовков, абзацев и списков;
- соблюдайте читаемость и структуру текста;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с HTML являются неправильное использование заголовков, пропущенные закрывающие теги, нарушение структуры текста, лишние переносы строк и некорректное оформление списков. Такие недочёты могут привести к ухудшению читаемости страницы, снижению доступности контента и проблемам с отображением в разных браузерах. Особенно важно соблюдать иерархию заголовков и не пропускать уровни оформления. Также необходимо следить за корректным закрытием всех тегов и аккуратной структурой документа.

1. Откройте урок модуля «Работа с текстом в HTML».
2. Прочитайте теорию и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственный текстовый блок с использованием разных тегов.
5. Проверьте отображение текста и корректность структуры страницы.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the WebLearn platform interface. At the top is a purple navigation bar with links: Главная, Курсы, Редактор, Конструктор, Доп. задания, and a user profile 'admin'. Below the navigation bar is a breadcrumb trail: Курсы / Основы HTML / Работа с текстом в HTML. On the left is a sidebar menu for 'Основы HTML' with items: Введение в веб-разрабо..., Структура HTML-докуме..., Основные теги и атрибу..., Работа с текстом в HTML (selected), Списки в HTML, and Гиперссылки. The main content area is titled 'Работа с текстом в HTML' and has a green 'Выполнено' (Completed) button. The text explains that text is a basic element of any web page and that HTML provides special tags for creating headings, paragraphs, and highlighting important information. A blue box contains a note: 'Важно: правильно оформленный текст делает страницу более понятной и удобной для чтения.' Below this is a section titled 'Заголовки' (Headings) which states that tags from <h1> to <h6> are used for creating headings. It lists: <h1> — самый важный и крупный заголовок, and <h2> — <h6> — подзаголовки разных уровней. At the bottom, a dark blue box shows example code: <h1>Главный заголовок</h1>, <h2>Подзаголовок</h2>, and <h3>Раздел статьи</h3>.

4.5. Модуль «Списки в HTML»

Цель модуля: научиться создавать и оформлять списки в HTML, использовать нумерованные, маркированные и вложенные списки для структурирования информации на веб-странице.

Рекомендуется проходить материал поэтапно и выполнять все практические задания. Сначала изучите теорию, затем разберите пример и повторите код вручную. После этого создайте собственный вариант списка и проверьте результат.

- перед началом задания определите, какой тип списка подходит для вашей страницы;
- добавляйте элементы списка постепенно и проверяйте отображение;
- используйте правильные теги для нумерованных и маркированных списков;
- соблюдайте вложенность и структуру элементов;
- после завершения выполните самопроверку по чек-листу.

Частые ошибки: неправильная вложенность списков, пропущенные закрывающие теги `</li>`, использование неверного типа списка и нарушение структуры HTML-документа. Проверяйте код после каждого изменения.

1. Откройте урок модуля «Списки в HTML».
2. Прочитайте теоретический материал и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственные маркированные, нумерованные и вложенные списки.
5. Проверьте корректность отображения и структуры списков.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the WebLearn platform interface. At the top is a navigation bar with links: Главная, Курсы, Редактор, Конструктор, Доп. задания, and a user profile 'admin'. Below the navigation bar, the breadcrumb path is 'Курсы / Основы HTML / Списки в HTML'. On the left, there is a sidebar menu with items: 'Основы HTML' (expanded), 'Введение в веб-разрабо...', 'Структура HTML-докуме...', 'Основные теги и атрибу...', 'Работа с текстом в HTML', 'Списки в HTML' (selected), and 'Гиперссылки'. The main content area is titled 'Списки в HTML' and includes a green 'Выполнено' (Completed) button. The text explains that lists are used on web pages to display information as a list of points. A blue box asks 'Где мы встречаем списки?' (Where do we encounter lists?) and lists examples: 'Меню сайта, инструкции, расписания, категории товаров, список задач и многое другое.' Below this, a section titled 'Виды списков в HTML' (Types of lists in HTML) states that there are two main types: 'Маркированный список' (List with markers) and 'Нумерованный список' (List with numbers). The 'Маркированный список' item is currently selected and highlighted with a blue diamond icon.

4.6. Модуль «Гиперссылки»

Цель модуля: научиться создавать и использовать гиперссылки в HTML, связывать страницы между собой и добавлять переходы на внешние ресурсы.

Рекомендуется изучать материал последовательно и выполнять практические задания после каждой темы. Сначала прочитайте теорию, затем разберите пример и повторите код вручную. После этого создайте собственные ссылки и проверьте их работу.

- перед началом задания определите, куда должна вести гиперссылка;
- добавляйте ссылки поэтапно и сразу проверяйте результат;
- используйте понятный текст ссылки для удобства пользователей;
- правильно указывайте адреса страниц и файлов;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе со ссылками в HTML являются неверно указанный путь к файлу, отсутствие обязательного атрибута href, незакрытые теги `<a>`, ошибки в написании адресов и неправильное использование внутренних ссылок внутри страницы. Такие недочёты могут привести к тому, что ссылки не будут работать или будут вести на неправильные страницы, что ухудшает пользовательский опыт.

1. Откройте урок модуля «Гиперссылки».
2. Прочитайте теоретический материал и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственные внутренние и внешние гиперссылки.
5. Проверьте корректность переходов и отображения ссылок.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the WebLearn platform interface. At the top is a purple navigation bar with links: Главная, Курсы, Редактор, Конструктор, and Доп. задания. A user profile icon labeled 'admin' is on the right. Below the navigation bar, the breadcrumb path is 'Курсы / Основы HTML / Гиперссылки'. On the left, a sidebar lists course topics: 'Основы HTML' (with sub-items: Введение в веб-разрабо..., Структура HTML-докуме..., Основные теги и атрибу..., Работа с текстом в HTML, Списки в HTML) and 'Гиперссылки' (highlighted). The main content area is titled 'Гиперссылки' and includes a green 'Выполнено' (Completed) button. The text explains that hyperlinks are one of the most important elements of web pages, allowing users to navigate between pages and access various internet resources. A blue box provides an example: 'Пример: когда вы нажимаете на кнопку «Подробнее», пункт меню или ссылку в поисковой системе, вы используете гиперссылку.' Below this, a section titled 'Что такое гиперссылка?' defines it as an element that allows navigating to a specified address. It states that the `<a>` tag is used for creating links. A code block shows the example: `<a href="https://www.google.com">Перейти на Google</a>`. A final note states: 'После нажатия пользователь перейдёт на указанный сайт.'

4.7. Модуль «Изображения»

Цель модуля: научиться добавлять изображения на веб-страницу, использовать атрибуты для настройки отображения и правильно размещать графические элементы в HTML-документе.

Рекомендуется проходить материал по порядку и выполнять все практические задания. Сначала изучите теорию, затем разберите пример и повторите код вручную. После этого добавьте собственные изображения и проверьте результат.

- перед началом задания определите, какие изображения будут использоваться на странице;
- добавляйте изображения постепенно и сразу проверяйте отображение;
- правильно указывайте путь к файлам изображений;
- используйте атрибуты alt, width и height при необходимости;
- соблюдайте структуру и читаемость кода;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с изображениями в HTML являются неправильный путь к файлу, отсутствие обязательного атрибута alt, ошибки в названии файлов, слишком большие размеры изображений и нарушение структуры страницы. Такие недочёты могут привести к тому, что изображение не отобразится, будет загружаться слишком долго или ухудшит доступность сайта для пользователей. Особенно важно корректно указывать пути к файлам и соблюдать регистр символов в их названиях. Атрибут alt необходим для описания изображения и улучшения доступности контента.

1. Откройте урок модуля «Изображения».
2. Прочитайте теорию и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Добавьте собственные изображения и настройте их параметры.
5. Проверьте корректность отображения изображений на странице.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the 'WebLearn' interface. At the top is a navigation bar with links: Главная, Курсы, Редактор, Конструктор, Доп. задания. A user profile 'admin' is in the top right. The main content area is titled 'Курсы / Продвинутый HTML / Изображения'. On the left is a sidebar with 'Продвинутый HTML' and a list of topics: 'Изображения' (selected), 'Таблицы', 'Формы', 'Встраивание мультимед...', 'Классы и идентификаторы...', and 'Семантические элементы...'. The main content area has a heading 'Изображения' and a green 'Выполнено' button. The text explains that images make web pages more attractive and informative. It includes a section 'Где используются изображения?' with the answer: 'В интернет-магазинах, социальных сетях, новостных сайтах, блогах и практически на любой современной веб-странице.' Below this is a section 'Тег изображения' explaining that the tag is used for adding images, with a code example: ``. The final note states: 'После загрузки страницы браузер отобразит указанное изображение.'

4.8. Модуль «Таблицы»

Цель модуля: научиться создавать таблицы в HTML, добавлять строки и столбцы, оформлять структуру данных и правильно использовать основные теги таблиц.

Рекомендуется изучать материал последовательно и выполнять практические задания после каждой темы. Сначала прочитайте теорию, затем разберите пример и повторите код вручную. После этого создайте собственную таблицу и проверьте результат.

- перед началом задания определите структуру таблицы и необходимые данные;
- добавляйте строки и столбцы постепенно, проверяя отображение;
- используйте правильные теги для таблиц, строк и ячеек;
- соблюдайте читаемость и аккуратную структуру кода;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при создании HTML-таблиц являются неправильная вложенность тегов `<table>`, `<tr>` и `<td>`, отсутствие обязательных закрывающих тегов, несоответствие количества ячеек в разных строках, а также нарушение общей логической структуры таблицы. Подобные ошибки могут привести к некорректному отображению данных в браузере, затруднить дальнейшую поддержку кода и вызвать проблемы при адаптации страницы под различные устройства.

1. Откройте урок модуля «Таблицы».
2. Прочитайте теоретический материал и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственную таблицу с несколькими строками и столбцами.
5. Проверьте корректность отображения и структуры таблицы.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the WebLearn interface. The top navigation bar includes links for 'Главная', 'Курсы', 'Редактор', 'Конструктор', and 'Доп. задания', along with a user profile 'admin'. The breadcrumb trail indicates the current location: 'Курсы / Продвинутый HTML / Таблицы'. A sidebar on the left lists course topics: 'Изображения', 'Таблицы' (selected), 'Формы', 'Встраивание мультимед...', 'Классы и идентификаторы...', and 'Семантические элементы...'. The main content area is titled 'Таблицы' and contains an introductory text about tables, a blue box with a practical example of a lesson schedule, and a section titled 'Основные теги таблиц' which includes a table of HTML tags.

Тег	Назначение
<code><table></code>	Создаёт таблицу
<code><tr></code>	Создаёт строку таблицы

4.9. Модуль «Формы»

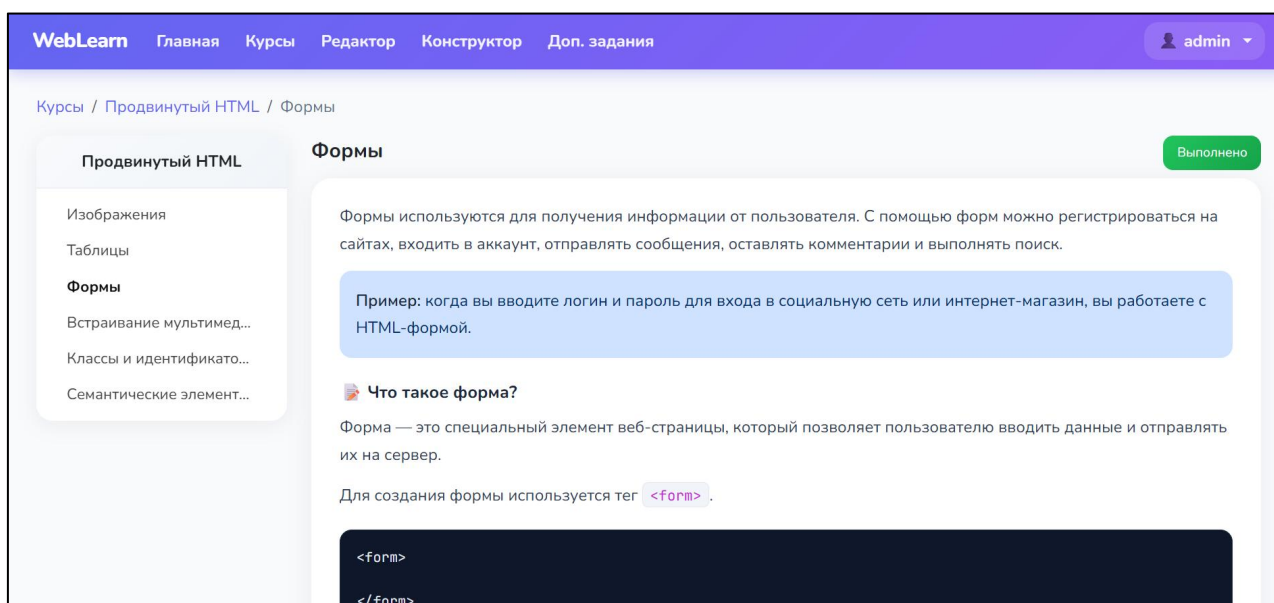
Цель модуля: научиться создавать формы в HTML, использовать поля ввода, кнопки, списки выбора и другие элементы для взаимодействия пользователя с веб-страницей.

Рекомендуется проходить материал по порядку и выполнять практические задания после изучения теории. Сначала прочитайте материал, затем разберите пример и повторите код вручную. После этого создайте собственную форму и проверьте её работу.

- перед началом задания определите, какие данные должна собирать форма;
- добавляйте элементы формы постепенно и сразу проверяйте результат;
- используйте подходящие типы полей ввода;
- подписывайте поля для удобства пользователя;
- соблюдайте структуру и читаемость кода;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с HTML-формами являются отсутствие атрибута `name`, неправильный выбор типа поля ввода, незакрытые теги, нарушение структуры формы и ошибки в размещении элементов. Такие недочёты могут привести к некорректной отправке данных или полной неработоспособности формы. Особенно важно правильно задавать атрибуты `name` и `type`, так как они определяют, какие данные и в каком формате будут передаваться. Также необходимо соблюдать корректную вложенность элементов формы и следить за закрытием всех тегов. Для предотвращения ошибок рекомендуется регулярно проверять код после каждого изменения и тестировать работу формы в браузере.

1. Откройте урок модуля «Формы».
2. Прочитайте теорию и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственную форму с несколькими полями ввода.
5. Проверьте корректность работы формы и отображение элементов.
6. Сохраните результат и при необходимости отправьте работу на проверку.



4.10. Модуль «Семантические элементы HTML5»

Цель модуля: научиться использовать семантические элементы HTML5 для правильной структуры веб-страницы и улучшения читаемости кода.

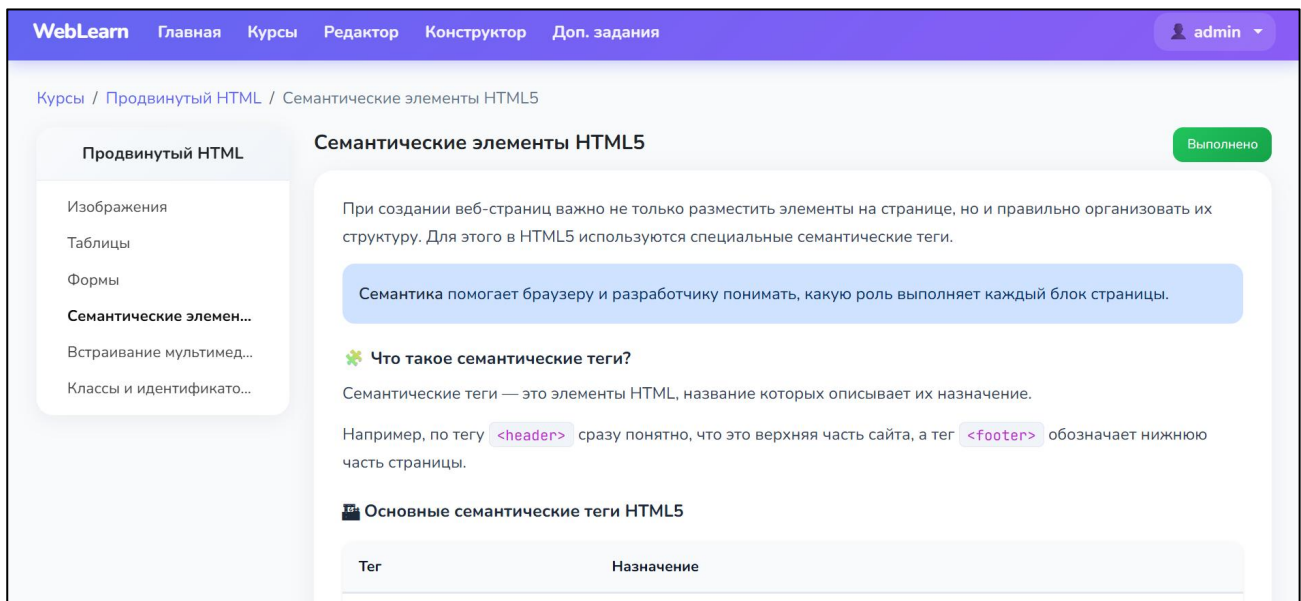
Рекомендуется изучать материал последовательно и выполнять все практические задания. Сначала прочитайте теорию, затем разберите пример и повторите код вручную. После этого создайте собственную структуру страницы с использованием семантических тегов.

- перед началом задания определите структуру страницы;
- используйте семантические теги по назначению;
- проверяйте отображение страницы после каждого изменения;
- соблюдайте логическую структуру документа;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при разработке HTML-страниц являются неправильное использование семантических тегов `<header>`, `<main>`, `<section>`, `<article>` и `<footer>`, а также нарушение логики их вложенности. Многие разработчики используют эти элементы без учёта их назначения, что приводит к ухудшению структуры документа, снижению доступности сайта и затрудняет его дальнейшую поддержку.

Чтобы избежать подобных ошибок, рекомендуется тщательно продумывать структуру документа ещё на этапе проектирования, использовать семантические элементы строго по их назначению и регулярно проверять валидность HTML-кода.

1. Откройте урок модуля «Семантические элементы HTML5».
2. Прочитайте теоретический материал и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственную структуру страницы с семантическими элементами.
5. Проверьте корректность отображения и структуры документа.
6. Сохраните результат и при необходимости отправьте работу на проверку.



4.11. Модуль «Встраивание мультимедиа»

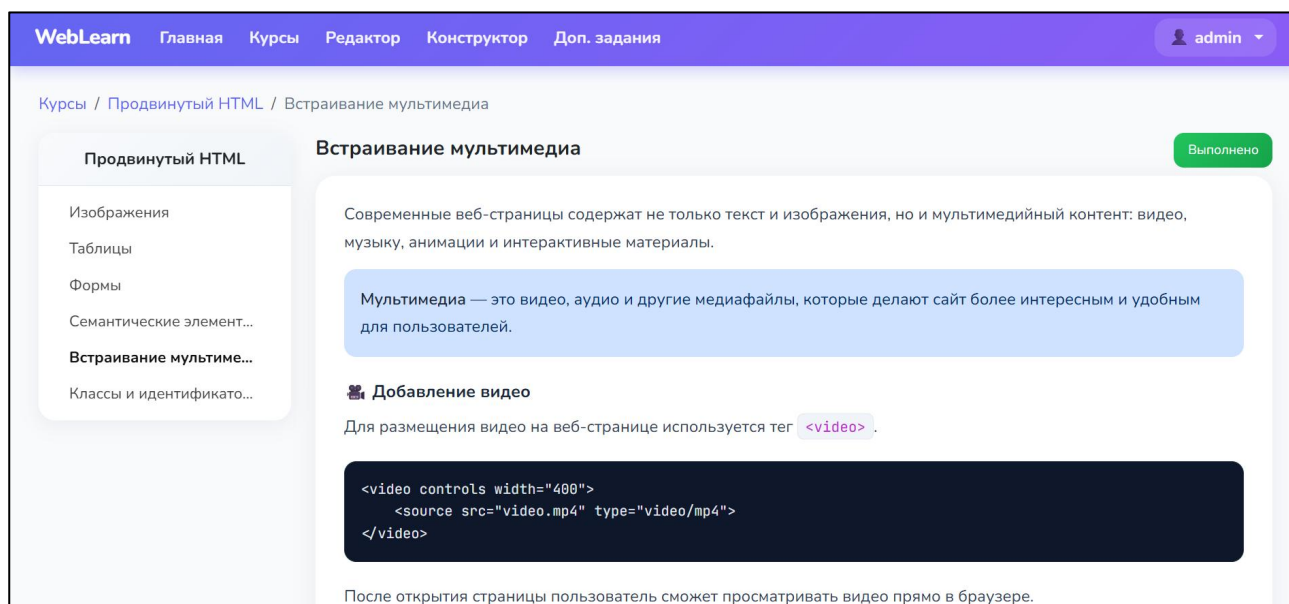
Цель модуля: научиться добавлять на веб-страницу аудио, видео и другие мультимедийные элементы с помощью HTML.

Рекомендуется проходить материал по порядку и выполнять практические задания после изучения теории. Сначала изучите пример, затем повторите код вручную и создайте собственный вариант страницы с мультимедиа.

- перед началом задания определите, какие мультимедийные элементы будут использоваться;
- добавляйте аудио и видео постепенно, проверяя результат;
- правильно указывайте пути к мультимедийным файлам;
- используйте элементы управления воспроизведением;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с мультимедийными элементами HTML являются неверное указание путей к файлам, использование неподдерживаемых форматов аудио и видео, отсутствие необходимых атрибутов управления, а также ошибки в структуре тегов `<audio>` и `<video>`. Подобные недочёты могут привести к тому, что мультимедийный контент не будет воспроизводиться на странице или будет работать некорректно в различных браузерах. Также важно учитывать кроссбраузерную совместимость и корректно задавать несколько источников через `<source>`, чтобы обеспечить стабильное воспроизведение. Для повышения надёжности рекомендуется проверять пути к файлам, использовать поддерживаемые форматы и тестировать работу мультимедиа после каждого изменения кода.

1. Откройте урок модуля «Встраивание мультимедиа».
2. Прочитайте теорию и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Добавьте собственные аудио- или видеофайлы на страницу.
5. Проверьте корректность воспроизведения мультимедиа.
6. Сохраните результат и при необходимости отправьте работу на проверку.



4.12. Модуль «Классы и идентификаторы»

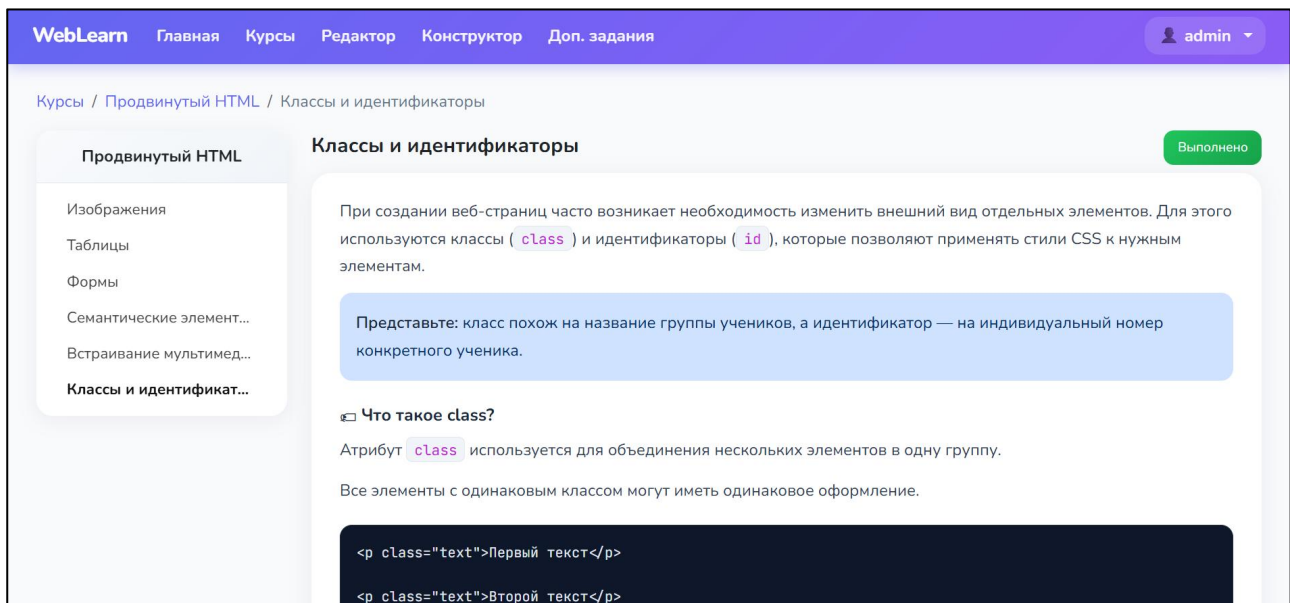
Цель модуля: научиться использовать классы и идентификаторы в HTML для стилизации элементов и организации структуры страницы.

Рекомендуется изучать материал последовательно и выполнять практические задания после каждой темы. Сначала внимательно прочитайте теоретический материал, затем изучите демонстрационный пример и повторите его самостоятельно без копирования. После этого создайте собственный вариант страницы, используя различные классы и идентификаторы для организации элементов.

- перед началом задания определите, какие элементы будут иметь классы и идентификаторы;
- используйте понятные и логичные названия;
- проверяйте результат после каждого изменения;
- соблюдайте структуру и читаемость кода;
- после завершения выполните самопроверку по чек-листу.

Частые ошибки при работе с классами и идентификаторами: повторение одинаковых идентификаторов для нескольких элементов страницы, использование слишком длинных или непонятных названий классов, ошибки в написании атрибутов `class` и `id`, использование пробелов и недопустимых символов в названиях, неправильное применение классов и идентификаторов, нарушение структуры HTML-документа, отсутствие проверки результата после внесения изменений, а также несоответствие названий классов в HTML и CSS.

1. Откройте урок модуля «Классы и идентификаторы».
2. Прочитайте теоретический материал и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Добавьте собственные классы и идентификаторы к элементам страницы.
5. Создайте уникальный элемент с использованием идентификатора.
6. Сохраните результат и при необходимости отправьте работу на проверку.



4.13. Модуль «Подключение CSS к HTML»

Цель модуля: научиться подключать CSS-стили к HTML-документу различными способами.

Рекомендуется проходить материал по порядку и выполнять практические задания после изучения теории. Сначала изучите пример, затем повторите код вручную и создайте собственный файл стилей.

- перед началом задания определите способ подключения CSS;
- проверяйте отображение страницы после каждого изменения;
- используйте отдельный CSS-файл для удобства работы;
- соблюдайте структуру HTML- и CSS-документов;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при подключении CSS являются неправильный путь к файлу, ошибки в теге `<link>`, отсутствие подключения стилей и нарушение структуры файлов проекта. Такие проблемы приводят к тому, что стили не применяются, и страница отображается без оформления или с некорректным дизайном. Особенно важно правильно указывать путь к CSS-файлу и проверять его расположение относительно HTML-документа. Также необходимо корректно использовать тег `<link>` в разделе `<head>`. Для предотвращения ошибок рекомендуется проверять подключение стилей после каждого изменения и тестировать отображение страницы в браузере.

1. Откройте урок модуля «Подключение CSS к HTML».
2. Прочитайте теорию и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Подключите собственный CSS-файл к HTML-документу.
5. Проверьте корректность отображения стилей.
6. Сохраните результат и при необходимости отправьте работу на проверку.

WebLearn Главная Курсы Редактор Конструктор Доп. задания  admin

Курсы / Изучение CSS / Подключение CSS к HTML

Изучение CSS
Подключение CSS к HTML...
Селекторы, классы и ид...
Цвета, текст и шрифты
Отступы, размеры и бло...
Position и display
Flexbox и CSS Grid

Подключение CSS к HTML

Выполнено

После создания структуры страницы с помощью HTML необходимо оформить её внешний вид. Для этого используется язык стилей CSS (Cascading Style Sheets).

CSS отвечает за внешний вид сайта: цвета, размеры текста, шрифты, отступы, расположение элементов и анимации.

 **Что такое CSS?**

CSS — это язык стилей, который позволяет изменять оформление HTML-элементов.

Без CSS веб-страницы выглядели бы как обычный текст без красивого дизайна.

 **Для чего используется CSS?**

- Изменение цвета текста
- Настройка размеров шрифта

4.14. Модуль «Селекторы, классы и идентификаторы»

Цель модуля: научиться использовать CSS-селекторы, классы и идентификаторы для стилизации элементов веб-страницы.

Рекомендуется изучать материал последовательно и выполнять практические задания после каждой темы. Сначала прочитайте теорию, затем разберите пример и повторите код вручную. После этого создайте собственные стили и проверьте результат.

- перед началом задания определите, какие элементы будут оформляться;
- используйте разные виды селекторов постепенно;
- применяйте понятные названия классов и идентификаторов;
- соблюдайте структуру и читаемость CSS-кода;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с CSS являются неправильное написание селекторов, использование одинаковых значений атрибута `id` для нескольких элементов, создание слишком сложных или неинформативных названий классов, а также ошибки при подключении таблиц стилей. Такие недочёты могут привести к некорректному отображению страницы, затруднить поддержку проекта и усложнить дальнейшую разработку.

Одной из самых распространённых проблем является неправильное написание селекторов. Ошибки в названиях классов, идентификаторов или тегов приводят к тому, что браузер не может применить нужные стили к элементам страницы.

1. Откройте урок модуля «Селекторы, классы и идентификаторы».
2. Прочитайте теорию и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Добавьте собственные классы и селекторы.
5. Проверьте корректность применения стилей.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the WebLearn platform interface. At the top is a purple navigation bar with links: Главная, Курсы, Редактор, Конструктор, and Доп. задания. On the right of the bar is a user profile icon labeled 'admin'. Below the navigation bar is a breadcrumb trail: Курсы / Изучение CSS / Селекторы, классы и идентификаторы. The main content area is divided into two columns. The left column, titled 'Изучение CSS', contains a list of topics: Подключение CSS к HT..., Селекторы, классы и и..., Цвета, текст и шрифты, Отступы, размеры и бло..., Position и display, and Flexbox и CSS Grid. The right column, titled 'Селекторы, классы и идентификаторы', has a green 'Выполнено' (Completed) button in the top right corner. The content in this column includes a paragraph explaining that after connecting CSS, one must specify which elements the styles will apply to using selectors. Below this is a blue box defining a selector as a rule that helps CSS find the right HTML element and change its appearance. Then, there is a section titled 'Что такое селектор?' (What is a selector?) with a subtext stating that a selector indicates which element needs to be styled. A code block shows a CSS rule:

```
p{
  color: blue;
}
```

. At the bottom, a note says: 'В данном примере все теги `<p>` будут отображаться синим цветом.'

4.15. Модуль «Цвета, текст и шрифты»

Цель модуля: научиться оформлять текст и элементы страницы с помощью цветов, шрифтов и CSS-свойств.

Рекомендуется изучать материал последовательно и выполнять практические задания после каждой темы. Сначала прочитайте теорию, затем разберите пример и повторите код вручную. После этого создайте собственное оформление страницы.

- перед началом задания определите цветовую схему страницы;
- изменяйте стили текста постепенно и проверяйте результат;
- используйте читаемые шрифты и размеры текста;
- соблюдайте единый стиль оформления;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при оформлении веб-страниц являются использование слишком ярких или конфликтующих цветов, создание нечитаемого текста, неправильное подключение шрифтов и чрезмерное количество стилевых правил. Такие недостатки негативно влияют на восприятие интерфейса пользователями, снижают удобство работы с сайтом и могут ухудшить общее впечатление от веб-ресурса.

Одной из наиболее заметных проблем является выбор слишком яркой цветовой палитры. Избыточно насыщенные цвета, особенно при использовании большого количества контрастных оттенков одновременно, могут утомлять зрение и отвлекать внимание от основного содержимого страницы.

1. Откройте урок модуля «Цвета, текст и шрифты».
2. Прочитайте теорию и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственное оформление текста и элементов.
5. Проверьте читаемость и отображение страницы.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the WebLearn platform interface. At the top is a purple navigation bar with links: Главная, Курсы, Редактор, Конструктор, and Доп. задания. A user profile 'admin' is in the top right. Below the navigation bar, the breadcrumb path is 'Курсы / Изучение CSS / Цвета, текст и шрифты'. On the left is a sidebar titled 'Изучение CSS' with a list of topics: 'Подключение CSS к HTML...', 'Селекторы, классы и идентификаторы...', 'Цвета, текст и шрифты' (highlighted), 'Отступы, размеры и блоки...', 'Position и display', and 'Flexbox и CSS Grid'. The main content area is titled 'Цвета, текст и шрифты' and has a green 'Выполнено' (Completed) button in the top right corner. The text explains that one of the main tasks of CSS is formatting text on a web page. A light blue box states: 'Без CSS весь текст выглядит одинаково. CSS помогает сделать страницу красивой, удобной и современной.' Below this, a section titled 'Изменение цвета текста' (Changing text color) explains that the 'color' property is used. A code block shows the CSS rule:

```
p{
  color: blue;
}
```

 A note at the bottom states: 'В данном примере весь текст внутри тегов <p> будет отображаться синим цветом.'

4.16. Модуль «Отступы, размеры и блочная модель»

Цель модуля: научиться управлять размерами элементов, внутренними и внешними отступами с помощью CSS.

Рекомендуется проходить материал поэтапно и выполнять все практические задания. Сначала внимательно изучите теоретический материал, затем разберите демонстрационный пример и повторите его самостоятельно без копирования. После этого создайте собственный макет страницы и попробуйте самостоятельно настроить размеры и расположение элементов.

- перед началом задания определите размеры и расположение элементов;
- изменяйте отступы постепенно и проверяйте результат;
- используйте свойства `margin`, `padding`, `width` и `height`;
- соблюдайте аккуратную структуру страницы;
- после завершения выполните самопроверку по чек-листу.

Частые ошибки: перепутанные внутренние и внешние отступы, неправильные размеры блоков, переполнение контента и нарушение структуры страницы. Проверьте код после каждого изменения.

1. Откройте урок модуля «Отступы, размеры и блочная модель».
2. Прочитайте теоретический материал и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Настройте размеры и отступы предложенных элементов.
5. Создайте собственный макет страницы с несколькими блоками.
6. Измените размеры элементов с помощью свойств `width` и `height`.
7. Настройте внутренние и внешние отступы с использованием `margin` и `padding`.
8. Проверьте корректность расположения блоков на странице.
9. Выполните самопроверку и убедитесь в отсутствии ошибок.
10. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the WebLearn platform interface. At the top is a purple navigation bar with links: Главная, Курсы, Редактор, Конструктор, and Доп. задания. A user profile 'admin' is in the top right. Below the navigation bar is a breadcrumb trail: Курсы / Изучение CSS / Отступы, размеры и блочная модель. On the left is a sidebar titled 'Изучение CSS' with a list of topics: Подключение CSS к HT..., Селекторы, классы и ид..., Цвета, текст и шрифты, **Отступы, размеры и бл...** (highlighted), Position и display, and Flexbox и CSS Grid. The main content area is titled 'Отступы, размеры и блочная модель' and has a green 'Выполнено' (Completed) button in the top right. The text explains that each element on a web page occupies a specific place and has its own dimensions, and that CSS can be used to control width, height, margins, and boundaries. A blue box contains the text: 'Представьте коробку: у неё есть содержимое, внутренние отступы, рамка и расстояние до других коробок. Именно так работает блочная модель в CSS.' Below this is a section titled 'Ширина и высота элемента' (Width and height of the element) with the text: 'Для изменения размеров используются свойства `width` и `height`.' A dark blue code block shows the CSS code:

```
div{
  width: 300px;
  height: 150px;
}
```

4.17. Модуль «Position и display»

Цель модуля: научиться управлять отображением и расположением элементов на веб-странице с помощью свойств `position` и `display`.

Рекомендуется изучать материал последовательно и выполнять практические задания после каждой темы. Сначала прочитайте теорию, затем разберите пример и повторите код вручную. После этого создайте собственное расположение элементов.

- перед началом задания определите расположение элементов на странице;
- изменяйте свойства постепенно и проверяйте результат;
- используйте разные значения `position` и `display`;
- соблюдайте структуру и читаемость CSS-кода;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с позиционированием элементов являются неправильное использование свойств `position: absolute` и `position: relative`, перекрытие элементов, нарушение структуры страницы и неточное размещение объектов. Из-за этого элементы могут отображаться не в том месте, выходить за пределы контейнера или закрывать друг друга. Особенно часто проблемы возникают при отсутствии понимания связи между родительскими и дочерними элементами. Чтобы избежать подобных ошибок, необходимо внимательно настраивать позиционирование, проверять расположение блоков и тестировать страницу после каждого внесённого изменения.

1. Откройте урок модуля «Position и display».
2. Прочитайте теорию и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственное расположение блоков на странице.
5. Проверьте корректность отображения элементов.
6. Сохраните результат и при необходимости отправьте работу на проверку.

WebLearn Главная Курсы Редактор Конструктор Доп. задания admin

Курсы / Изучение CSS / Position и display

Изучение CSS

- Подключение CSS к НТ...
- Селекторы, классы и ид...
- Цвета, текст и шрифты
- Отступы, размеры и бло...
- Position и display**
- Flexbox и CSS Grid

Position и display Выполнено

При разработке веб-страниц важно не только оформлять элементы, но и правильно размещать их на странице. Для этого в CSS используются свойства `display` и `position`.

Display определяет, как элемент отображается на странице, а position отвечает за его расположение.

Свойство display

Свойство `display` управляет способом отображения элемента.

Значение	Назначение
<code>block</code>	Элемент занимает всю доступную ширину строки
<code>inline</code>	Элемент располагается в строке с другими элементами
<code>flex</code>	Гибкое расположение элементов

4.18. Модуль «Flexbox и CSS Grid»

Цель модуля: научиться создавать современные макеты веб-страниц с помощью Flexbox и CSS Grid.

Рекомендуется проходить материал по порядку и выполнять практические задания после изучения теории. Сначала изучите пример, затем повторите код вручную и создайте собственный макет страницы.

- перед началом задания определите структуру будущего макета;
- используйте Flexbox для выравнивания элементов;
- применяйте CSS Grid для создания сетки страницы;
- проверяйте расположение элементов после каждого изменения;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с Flexbox и Grid являются неправильное использование их свойств, нарушение структуры сетки, переполнение блоков контентом и отсутствие адаптивности. Такие ошибки могут привести к некорректному расположению элементов на странице и ухудшению пользовательского опыта. Нередко разработчики неверно задают размеры колонок и строк, что вызывает смещение или наложение элементов. Также важно учитывать отображение сайта на разных устройствах и экранах. Для предотвращения проблем рекомендуется регулярно проверять код, тестировать макет и анализировать результат после каждого изменения.

1. Откройте урок модуля «Flexbox и CSS Grid».
2. Прочитайте теорию и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственный макет страницы с использованием Flexbox или Grid.
5. Проверьте корректность расположения элементов.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the WebLearn interface. At the top is a purple navigation bar with links: Главная, Курсы, Редактор, Конструктор, Доп. задания, and a user profile 'admin'. Below the navigation bar is a breadcrumb trail: Курсы / Изучение CSS / Flexbox и CSS Grid. On the left is a sidebar titled 'Изучение CSS' with a list of topics: Подключение CSS к HTML, Селекторы, классы и идентификаторы, Цвета, текст и шрифты, Отступы, размеры и блоки, Position и display, and Flexbox и CSS Grid (which is highlighted). The main content area is titled 'Flexbox и CSS Grid' and has a green 'Выполнено' (Completed) button in the top right corner. The text in the main area explains that for modern websites, it's important to correctly place elements on the page, and that CSS uses two powerful technologies: Flexbox and CSS Grid. A blue box states: 'Flexbox удобно использовать для расположения элементов в одну строку или колонку, а Grid идеально подходит для создания сложных сеток и макетов.' Below this, a section titled 'Что такое Flexbox?' explains that Flexbox (Flexible Box Layout) is a CSS technology that helps easily align and place elements within a container. At the bottom, a code block shows the CSS for a container:

```
.container{
  display: flex;
}
```

4.19. Модуль «Введение в JavaScript»

Цель модуля: познакомиться с языком JavaScript и научиться создавать простые интерактивные элементы на веб-странице.

Рекомендуется изучать материал последовательно и выполнять практические задания после каждой темы. Сначала прочитайте теорию, затем разберите пример и повторите код вручную. После этого создайте собственный скрипт и проверьте результат.

- перед началом задания определите, что должен выполнять скрипт;
- вводите код постепенно и сразу проверяйте результат;
- соблюдайте структуру и читаемость кода;
- используйте понятные имена переменных;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с JavaScript являются пропущенные скобки, ошибки в написании команд, неправильное подключение скриптов и нарушение синтаксиса. Такие недочёты приводят к тому, что код перестаёт выполняться или работает с ошибками, что затрудняет взаимодействие пользователя с веб-страницей. Особенно важно внимательно следить за правильностью написания функций, переменных и операторов. Также необходимо проверять корректность подключения JavaScript-файлов к HTML-документу. Для предотвращения ошибок рекомендуется тестировать код после каждого изменения и использовать инструменты разработчика для поиска и исправления проблем.

1. Откройте урок модуля «Введение в JavaScript».
2. Прочитайте теорию и изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственный простой скрипт.
5. Проверьте корректность работы программы.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot displays the WebLearn platform interface. At the top, a purple navigation bar contains links: Главная, Курсы, Редактор, Конструктор, and Доп. задания. A user profile icon labeled 'admin' is on the right. Below the navigation bar, the breadcrumb path is 'Курсы / Изучение JavaScript / Введение в JavaScript'. The main content area is divided into two columns. The left column, titled 'Изучение JavaScript', contains a list of topics: 'Введение в JavaScript', 'Подключение JavaScript ...', 'Переменные и типы дан...', 'Операторы и выражения', 'Обработка событий', 'Циклы', 'Функции', 'Условные конструкции', 'Работа с DOM', 'Валидация форм с помо...', and 'Итоговый проект (интер...'. The right column, titled 'Введение в JavaScript', features a green 'Выполнено' (Completed) button. The text explains that JavaScript is a programming language that makes web pages interactive. A blue box highlights that HTML creates structure, CSS handles appearance, and JavaScript adds actions and interactivity. Below this, a section titled 'Что умеет JavaScript?' (What JavaScript can do) lists capabilities: changing text and images, handling button clicks, and creating animations.

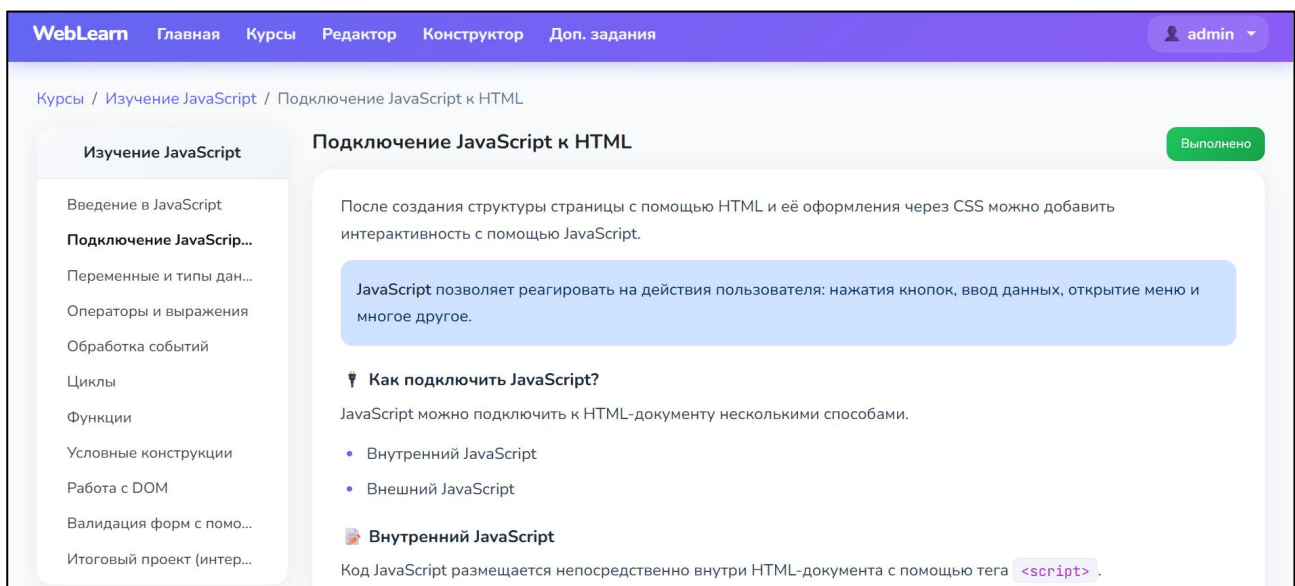
4.20. Модуль «Подключение JavaScript к HTML»

Цель модуля — научиться подключать JavaScript к HTML-документу и запускать скрипты на веб-странице. В процессе обучения рассматриваются основные способы подключения JavaScript, включая использование тега `<script>` внутри HTML-файла и подключение внешних файлов. Также уделяется внимание правильному размещению скриптов в структуре страницы и особенностям их выполнения в браузере. Освоение данного модуля позволяет понять базовые принципы работы JavaScript и использовать его для добавления интерактивности на веб-страницах.

- используйте тег `<script>` правильно;
- проверяйте подключение файла после каждого изменения;
- соблюдайте структуру HTML- и JS-файлов;
- используйте отдельный JS-файл для удобства работы;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при подключении JavaScript являются неправильный путь к JS-файлу, ошибки в теге `<script>`, отсутствие подключения скрипта и нарушение структуры файлов проекта. Такие недочёты приводят к тому, что скрипты не выполняются, а интерактивные элементы страницы перестают работать. Особенно важно правильно указывать путь к файлу и проверять его расположение относительно HTML-документа. Также необходимо корректно размещать тег `<script>` в структуре страницы и следить за его закрытием. Для предотвращения ошибок рекомендуется проверять подключение скриптов после каждого изменения и тестировать работу функционала в браузере.

1. Откройте урок модуля «Подключение JavaScript к HTML».
2. Изучите пример подключения скрипта.
3. Повторите пример в редакторе без копирования.
4. Подключите собственный JavaScript-файл.
5. Проверьте корректность работы скрипта.
6. Сохраните результат и при необходимости отправьте работу на проверку.



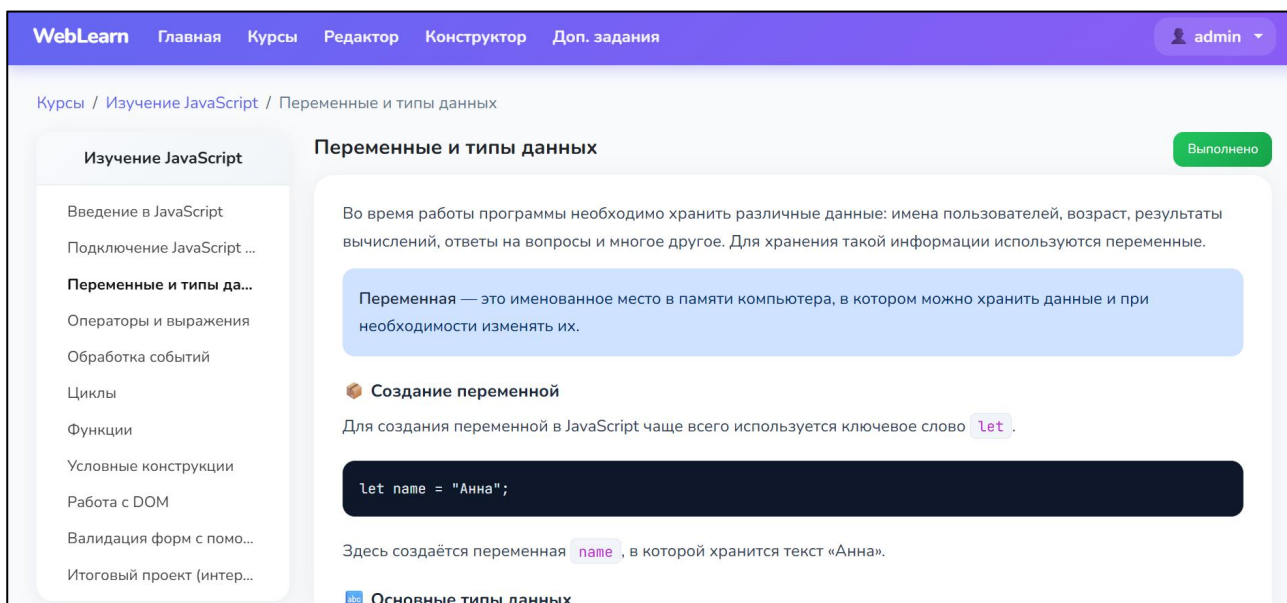
4.21. Модуль «Переменные и типы данных»

Цель модуля — научиться создавать переменные и использовать основные типы данных в JavaScript. В ходе изучения рассматриваются способы объявления переменных с помощью `let`, `const` и `var`, а также особенности их применения в разных ситуациях. Отдельное внимание уделяется основным типам данных, таким как числа, строки, логические значения, `null`, `undefined` и объекты. Освоение этого модуля позволяет понять, как хранить и обрабатывать информацию в программе, а также формирует базовые навыки работы с данными в JavaScript, необходимые для дальнейшего изучения языка.

- используйте понятные имена переменных;
- проверяйте значения переменных после каждого изменения;
- соблюдайте синтаксис JavaScript;
- различайте типы данных;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с JavaScript являются неправильное объявление переменных, путаница между типами данных и ошибки в написании команд. Такие недочёты могут привести к некорректной работе скриптов, неожиданным результатам выполнения кода или полному его сбою. Особенно важно внимательно использовать ключевые слова `let`, `const` и `var`, а также понимать различия между типами данных, такими как строки, числа и логические значения. Для предотвращения ошибок рекомендуется тщательно проверять синтаксис, тестировать код после каждого изменения и использовать инструменты разработчика для поиска и исправления проблем.

1. Откройте урок модуля «Переменные и типы данных».
2. Прочитайте теорию и изучите пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственные переменные разных типов.
5. Проверьте корректность вывода данных.
6. Сохраните результат и при необходимости отправьте работу на проверку.



4.22. Модуль «Операторы и выражения»

Цель модуля — научиться использовать арифметические, логические и сравнительные операторы в JavaScript. В процессе обучения рассматриваются основные виды операторов, их назначение и особенности применения в различных ситуациях. Арифметические операторы позволяют выполнять математические вычисления, сравнительные — сравнивать значения и получать логический результат, а логические операторы помогают строить сложные условия и управлять логикой программы. Освоение данного модуля помогает понять, как формируются выражения в JavaScript и как с их помощью можно управлять поведением кода.

- выполняйте вычисления поэтапно;
- проверяйте результат выражений;
- соблюдайте синтаксис операторов;
- используйте комментарии для сложных выражений;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с операторами в JavaScript являются неправильный порядок вычислений, путаница между операторами сравнения и присваивания, а также ошибки в логических выражениях. Такие недочёты могут приводить к неожиданным результатам выполнения кода или нарушению логики программы. Особенно важно понимать разницу между `=` и `==` или `===`, а также правильно использовать скобки для задания приоритета операций. Логические выражения требуют внимательного построения условий, чтобы избежать ошибок в принятии решений программой.

1. Откройте урок модуля «Операторы и выражения».
2. Изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственные выражения и вычисления.
5. Проверьте корректность результатов.
6. Сохраните результат и при необходимости отправьте работу на проверку.

WebLearn
Главная
Курсы
Редактор
Конструктор
Доп. задания
admin

Курсы / Изучение JavaScript / Операторы и выражения

Изучение JavaScript

Введение в JavaScript
Подключение JavaScript ...
Переменные и типы дан...
Операторы и выражения
Обработка событий
Циклы
Функции
Условные конструкции
Работа с DOM
Валидация форм с помо...
Итоговый проект (интер...

Операторы и выражения

Выполнено

При создании программ часто требуется выполнять вычисления, сравнивать значения и проверять различные условия. Для этого в JavaScript используются операторы.

Оператор — это специальный символ или команда, которая выполняет определённое действие над данными.

Математические операторы

Математические операторы используются для выполнения вычислений.

Оператор	Назначение	Пример
+	Сложение	5 + 3
-	Вычитание	10 - 4
*	Умножение	6 * 2

4.23. Модуль «Обработка событий»

Цель модуля: научиться реагировать на действия пользователя с помощью событий JavaScript. В ходе изучения модуля обучающиеся познакомятся с основными типами событий, такими как клики, ввод текста, наведение курсора и изменение элементов интерфейса. Особое внимание уделяется практическому применению обработчиков событий для создания интерактивных веб-страниц. Также рассматриваются способы передачи данных из событий, управление поведением элементов и динамическое изменение содержимого страницы в ответ на действия пользователя.

- определите, какие действия должен выполнять пользователь;
- проверяйте события после каждого изменения;
- используйте обработчики событий правильно;
- соблюдайте структуру кода;
- после завершения выполните самопроверку по чек-листу.

Частые ошибки: неправильное подключение событий, ошибки обработчиков и неверное обращение к элементам страницы. Также распространённой проблемой является использование несуществующих или неверно выбранных селекторов, из-за чего скрипт не может найти нужный элемент на странице. Ошибки возникают и при неправильной работе с контекстом внутри обработчиков событий, а также при отсутствии проверки на существование элемента перед его использованием. Кроме того, обучающиеся часто забывают удалять или корректно переопределять обработчики событий, что может приводить к неожиданному поведению интерфейса.

1. Откройте урок модуля «Обработка событий».
2. Изучите демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Добавьте собственные обработчики событий.
5. Проверьте корректность работы интерактивных элементов.
6. Сохраните результат и при необходимости отправьте работу на проверку.

WebLearn Главная Курсы Редактор Конструктор Доп. задания  admin

Курсы / Изучение JavaScript / Обработка событий

Изучение JavaScript
Введение в JavaScript
Подключение JavaScript ...
Переменные и типы дан...
Операторы и выражения
Обработка событий
Циклы
Функции
Условные конструкции
Работа с DOM
Валидация форм с помо...
Итоговый проект (интер...

Обработка событий

Выполнено

Одной из главных возможностей JavaScript является обработка событий. Благодаря событиям веб-страница может реагировать на действия пользователя и выполнять различные команды.

Событие — это действие пользователя или браузера, на которое программа может отреагировать.

🔗 Что такое событие?

Когда пользователь нажимает кнопку, наводит курсор мыши или отправляет форму, происходит событие. JavaScript может отслеживать эти действия и выполнять нужный код.

👉 Событие click

Событие `click` происходит при нажатии на элемент.

```
<button onClick="showText()">  
Нажми меня
```

4.24. Модуль «Циклы»

Цель модуля: научиться использовать циклы для многократного выполнения команд в JavaScript. В рамках изучения модуля обучающиеся осваивают основные конструкции циклов, такие как `for`, `while` и `do...while`, а также учатся выбирать наиболее подходящий тип цикла в зависимости от задачи. Особое внимание уделяется правильной организации условий выполнения и завершения цикла, чтобы избежать ошибок и неоптимального кода.

- определите количество повторений;
- проверяйте работу цикла после каждого изменения;
- используйте подходящий тип цикла;
- избегайте бесконечных циклов;
- после завершения выполните самопроверку по чек-листу.

Частые ошибки: бесконечные циклы, неправильные условия остановки, ошибки в работе счётчиков, а также изменение переменной цикла в некорректных местах, что приводит к неожиданному поведению программы. Дополнительно встречаются случаи, когда условие выхода из цикла задано неверно или не учитывает все возможные значения. Нередко обучающиеся используют неправильные операторы сравнения, из-за чего цикл выполняется лишнее количество раз или, наоборот, завершается раньше времени. Также встречаются ошибки при работе с вложенными циклами, когда внутренний цикл влияет на переменные внешнего, что нарушает логику программы. Ещё одной распространённой проблемой является отсутствие контроля за изменением счётчика, особенно в циклах `while`, что приводит к зависанию программы.

1. Откройте урок модуля «Циклы».
2. Изучите теорию и демонстрационный пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственный цикл.
5. Проверьте корректность выполнения программы.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the 'WebLearn' platform interface. The top navigation bar includes links for 'Главная', 'Курсы', 'Редактор', 'Конструктор', and 'Доп. задания', along with a user profile 'admin'. The main content area is titled 'Курсы / Изучение JavaScript / Циклы'. On the left, a sidebar lists course topics, with 'Циклы' highlighted. The main content area has a green 'Выполнено' (Completed) button. The text explains that cycles are used to repeat actions. A definition states: 'Цикл — это конструкция, которая позволяет многократно выполнять один и тот же блок кода.' A section titled 'Зачем нужны циклы?' explains that writing 100 `console.log()` commands is inconvenient, and a cycle is a simpler solution. It then introduces the 'for' cycle, stating it is used when the number of repetitions is known in advance. A code snippet is shown:

```
for(let i = 1; i <= 5; i++){
```

4.25. Модуль «Функции»

Цель модуля — научиться создавать и использовать функции для организации и повторного использования кода в JavaScript. В процессе изучения рассматриваются основы объявления функций, передача параметров и возврат значений. Функции позволяют структурировать программу, избегать дублирования кода и делать его более читаемым и удобным для поддержки. Освоение данного модуля помогает понять принципы модульности программирования и применять функции для решения различных практических задач.

- используйте понятные имена функций;
- разделяйте код на небольшие функции;
- проверяйте работу функций после каждого изменения;
- соблюдайте структуру кода;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с функциями в JavaScript являются неправильный вызов функций, отсутствие необходимых параметров и ошибки при возврате значений. Такие недочёты могут привести к тому, что функция не выполнит свою задачу или вернёт некорректный результат. Особенно важно правильно передавать аргументы при вызове функции и внимательно следить за использованием оператора `return`, так как он определяет итоговое значение. Для предотвращения ошибок рекомендуется проверять соответствие параметров, тестировать функции на разных входных данных и использовать отладку для анализа работы кода.

1. Откройте урок модуля «Функции».
2. Прочитайте теорию и изучите пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственные функции.
5. Проверьте корректность работы программы.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the WebLearn platform interface. At the top is a navigation bar with links: Главная, Курсы, Редактор, Конструктор, Доп. задания, and a user profile 'admin'. Below the navigation bar, the breadcrumb path is 'Курсы / Изучение JavaScript / Функции'. The main content area is divided into two columns. The left column, titled 'Изучение JavaScript', contains a list of topics: Введение в JavaScript, Подключение JavaScript..., Переменные и типы дан..., Операторы и выражения, Обработка событий, Циклы, **Функции** (highlighted), Условные конструкции, Работа с DOM, Валидация форм с помо..., and Итоговый проект (интер...). The right column, titled 'Функции', contains the following text: 'При разработке программ часто возникает необходимость выполнять одни и те же действия несколько раз. Чтобы не писать одинаковый код повторно, используются функции.' Below this is a blue box with the definition: 'Функция — это именованный блок кода, который выполняет определённую задачу и может вызываться многократно.' Further down is a section '✂ Зачем нужны функции?' with the text: 'Функции делают код более удобным, понятным и позволяют избежать повторений. Вместо того чтобы несколько раз писать одинаковые команды, можно создать функцию и вызывать её по мере необходимости.' At the bottom is a section '🎨 Создание функции' with the text: 'Для создания функции используется ключевое слово `function`.'

4.26. Модуль «Условные конструкции»

Цель модуля — научиться использовать условные конструкции `if`, `else` и `switch` для принятия решений в программе на JavaScript. В ходе изучения рассматриваются принципы построения условий, особенности их работы и выбор подходящей конструкции в зависимости от задачи. Условные операторы позволяют управлять логикой выполнения кода, выполнять разные действия при различных значениях переменных и создавать более гибкие программы. Освоение данного модуля помогает понять, как программа реагирует на разные ситуации и как правильно организовать ветвление логики.

- определите условия выполнения программы;
- проверяйте результат после каждого изменения;
- используйте правильную структуру условий;
- соблюдайте читаемость кода;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при использовании условных конструкций `if`, `else` и `switch` в JavaScript являются неправильно заданные условия, пропущенные фигурные скобки и ошибки в логических выражениях. Такие недочёты могут привести к тому, что программа будет выполнять не те действия, которые ожидаются, или вовсе не выполнит нужный блок кода. Особенно важно внимательно проверять синтаксис условий, корректно группировать выражения и соблюдать структуру блоков кода. Для конструкции `switch` необходимо следить за правильным использованием `case` и `break`, чтобы избежать случайного выполнения лишних веток.

1. Откройте урок модуля «Условные конструкции».
2. Прочитайте теорию и изучите пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственные условия в программе.
5. Проверьте корректность выполнения кода.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot displays the WebLearn platform interface. At the top, a purple navigation bar contains links: Главная, Курсы, Редактор, Конструктор, and Доп. задания. A user profile icon labeled 'admin' is on the right. Below the navigation bar, the breadcrumb path is 'Курсы / Изучение JavaScript / Условные конструкции'. The main content area is divided into two columns. The left column, titled 'Изучение JavaScript', lists various topics: Введение в JavaScript, Подключение JavaScript..., Переменные и типы дан..., Операторы и выражения, Обработка событий, Циклы, Функции, Условные конструкции (highlighted), Работа с DOM, Валидация форм с помо..., and Итоговый проект (интер...). The right column, titled 'Условные конструкции', contains a green 'Выполнено' button, an introductory text about conditional structures, a definition of a condition, a section titled 'Что такое оператор if?' with an emoji, and a code example:

```
let age = 18;

if (age >= 18) {
  alert("Совершеннолетний");
}
```

4.27. Модуль «Работа с DOM»

Цель модуля — научиться взаимодействовать с HTML-элементами страницы через DOM с помощью JavaScript. В процессе изучения рассматриваются основные принципы работы с DOM-структурой, способы поиска элементов на странице, а также методы изменения их содержимого, атрибутов и стилей. Освоение данного модуля позволяет управлять элементами веб-страницы динамически, реагировать на действия пользователя и создавать интерактивные интерфейсы. Понимание работы с DOM является важной основой для дальнейшего изучения JavaScript и разработки современных веб-приложений.

- выбирайте элементы страницы правильно;
- проверяйте изменения после каждого действия;
- используйте методы DOM аккуратно;
- соблюдайте структуру кода;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при работе с DOM в JavaScript являются неправильные селекторы, ошибки при изменении элементов и неверное обращение к DOM-объектам. Такие недочёты могут привести к тому, что нужные элементы не будут найдены или изменения не отобразятся на странице. Особенно важно внимательно проверять корректность использования методов поиска, таких как `getElementById`, `querySelector` и `querySelectorAll`, а также учитывать, что некоторые методы возвращают коллекции элементов. Для предотвращения ошибок рекомендуется тестировать код после каждого изменения, проверять структуру HTML и использовать инструменты разработчика для отладки.

1. Откройте урок модуля «Работа с DOM».
2. Изучите теорию и пример.
3. Повторите пример в редакторе без копирования.
4. Измените элементы страницы через JavaScript.
5. Проверьте корректность работы.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot shows the WebLearn platform interface. At the top is a navigation bar with links: Главная, Курсы, Редактор, Конструктор, Доп. задания, and a user profile 'admin'. Below the navigation bar, the breadcrumb path is 'Курсы / Изучение JavaScript / Работа с DOM'. The main content area is divided into two columns. The left column, titled 'Изучение JavaScript', contains a list of topics: Введение в JavaScript, Подключение JavaScript..., Переменные и типы дан..., Операторы и выражения, Обработка событий, Циклы, Функции, Условные конструкции, Работа с DOM (highlighted), Валидация форм с помо..., and Итоговый проект (интер...). The right column, titled 'Работа с DOM', contains a green 'Выполнено' (Completed) button. Below the title, there is a text block explaining that after loading a web page, the browser creates a special structure called DOM, and JavaScript can find elements, change their content, styles, and behavior. A blue box defines DOM (Document Object Model) as an object model of the document representing the HTML page as a collection of objects and elements. Below this, a green tree icon is followed by the heading 'Что такое DOM?'. The text explains that when the browser opens an HTML document, it turns all page elements into objects that JavaScript can work with. At the bottom, a dark blue code block shows a snippet of HTML: `<body>`, `<h1>Заголовок</h1>`, and `<p>Текст</p>`.

4.28. Модуль «Валидация форм с помощью JavaScript»

Цель модуля — научиться проверять корректность введённых данных в формах с помощью JavaScript. В процессе изучения рассматриваются основные методы валидации пользовательского ввода, включая проверку на пустые значения, соответствие формату и допустимым диапазонам. Также уделяется внимание обработке ошибок и отображению сообщений пользователю. Освоение данного модуля позволяет повысить надёжность веб-форм, предотвратить отправку некорректных данных и улучшить пользовательский опыт при взаимодействии с сайтом.

- определите правила проверки данных;
- проверяйте результат после каждого изменения;
- выводите понятные сообщения об ошибках;
- соблюдайте структуру кода;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при валидации форм с помощью JavaScript являются неправильная проверка полей, отсутствие сообщений об ошибках и неверная логика условий. Такие недочёты могут привести к отправке некорректных данных или отсутствию обратной связи для пользователя. Особенно важно корректно проверять обязательные поля, учитывать различные форматы ввода и правильно строить условные выражения. Также необходимо выводить понятные сообщения об ошибках, чтобы пользователь мог быстро исправить введённые данные.

1. Откройте урок модуля «Валидация форм с помощью JavaScript».
2. Изучите теорию и пример.
3. Повторите пример в редакторе без копирования.
4. Создайте собственную проверку формы.
5. Проверьте корректность работы валидации.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot displays the WebLearn platform interface. At the top, a purple navigation bar contains links: Главная, Курсы, Редактор, Конструктор, and Доп. задания. A user profile icon labeled 'admin' is in the top right. Below the navigation bar, the breadcrumb path reads: Курсы / Изучение JavaScript / Валидация форм с помощью JavaScript. The main content area is divided into two columns. The left column, titled 'Изучение JavaScript', lists various topics: Введение в JavaScript, Подключение JavaScript..., Переменные и типы дан..., Операторы и выражения, Обработка событий, Циклы, Функции, Условные конструкции, Работа с DOM, Валидация форм с пом..., and Итоговый проект (интер...). The right column, titled 'Валидация форм с помощью JavaScript', features a green 'Выполнено' (Completed) button in the top right corner. The text explains that users can make mistakes when filling out forms, such as leaving fields empty or entering incorrect email addresses or passwords. It states that validation is used to check the correctness of the entered data. A blue box contains the definition: 'Валидация — это процесс проверки данных перед их отправкой на сервер.' Below this, a section titled 'Зачем нужна валидация?' (Why is validation needed?) explains that validation helps prevent errors and ensures that the user has entered the necessary data correctly. A bulleted list follows: Проверка обязательных полей, Проверка электронной почты, Проверка пароля, and Проверка номера телефона.

4.29. Модуль «Итоговый проект (интерактивный сайт)»

Цель модуля — закрепить знания HTML, CSS и JavaScript при создании полноценного интерактивного веб-сайта. В процессе работы учащиеся объединяют полученные ранее навыки верстки, стилизации и программирования, чтобы разработать функциональный и визуально оформленный проект. Особое внимание уделяется взаимодействию всех технологий между собой, корректной структуре кода и организации файловой системы. Освоение данного модуля позволяет сформировать целостное понимание процесса веб-разработки и применить знания на практике при создании реального проекта.

- составьте план структуры проекта;
- разделяйте HTML, CSS и JavaScript по файлам;
- проверяйте работу сайта после каждого этапа;
- используйте понятные названия классов и функций;
- после завершения выполните самопроверку по чек-листу.

Частыми ошибками при создании интерактивного веб-сайта являются нарушение структуры проекта, ошибки взаимодействия HTML, CSS и JavaScript, отсутствие адаптивности и неправильная организация кода. Такие недочёты приводят к тому, что сайт работает нестабильно, элементы отображаются некорректно, а поддержка проекта становится сложной. Особенно важно правильно выстраивать связь между слоями приложения, соблюдать единый стиль кода и следить за логикой файловой структуры. Также необходимо учитывать адаптивность интерфейса для различных устройств.

1. Откройте урок модуля «Итоговый проект».
2. Изучите пример проекта.
3. Повторите основные элементы в редакторе.
4. Создайте собственный интерактивный сайт.
5. Проверьте корректность работы всех элементов.
6. Сохраните результат и при необходимости отправьте работу на проверку.

The screenshot displays the WebLearn platform interface. At the top, a purple navigation bar contains the 'WebLearn' logo and links for 'Главная', 'Курсы', 'Редактор', 'Конструктор', and 'Доп. задания'. A user profile icon labeled 'admin' is in the top right. Below the navigation bar, a breadcrumb trail reads 'Курсы / Изучение JavaScript / Итоговый проект (интерактивный сайт)'. The main content area is divided into two columns. The left column, titled 'Изучение JavaScript', lists various topics: 'Введение в JavaScript', 'Подключение JavaScript ...', 'Переменные и типы дан...', 'Операторы и выражения', 'Обработка событий', 'Циклы', 'Функции', 'Условные конструкции', 'Работа с DOM', 'Валидация форм с помо...', and 'Итоговый проект (инте...'. The right column, titled 'Итоговый проект (интерактивный сайт)', features a green 'Выполнено' (Completed) button in the top right corner. The text in this column states: 'На завершающем этапе изучения HTML, CSS и JavaScript учащиеся создают собственный интерактивный веб-сайт. Этот проект позволяет применить на практике все полученные знания и навыки веб-разработки.' Below this, a blue box contains the project goal: 'Цель проекта — создать современный веб-сайт, который имеет красивый дизайн, удобный интерфейс и интерактивные элементы.' Further down, a section titled 'Какие проекты можно создать?' (What projects can be created?) with a rocket icon explains that students can choose the theme of their future site. A list of project ideas follows: 'Персональный сайт-визитка', 'Сайт-портфолио', 'Мини-игра на JavaScript', and 'Форма регистрации пользователей'.

5. Практические сценарии работы

5.1. Создание личной страницы с блоком «О себе»

В каждом учебном сценарии представлена чётко сформулированная задача, последовательность шагов выполнения и критерии проверки результата. Такой подход позволяет структурировать процесс работы, снизить количество ошибок и обеспечить постепенное освоение материала на практике.

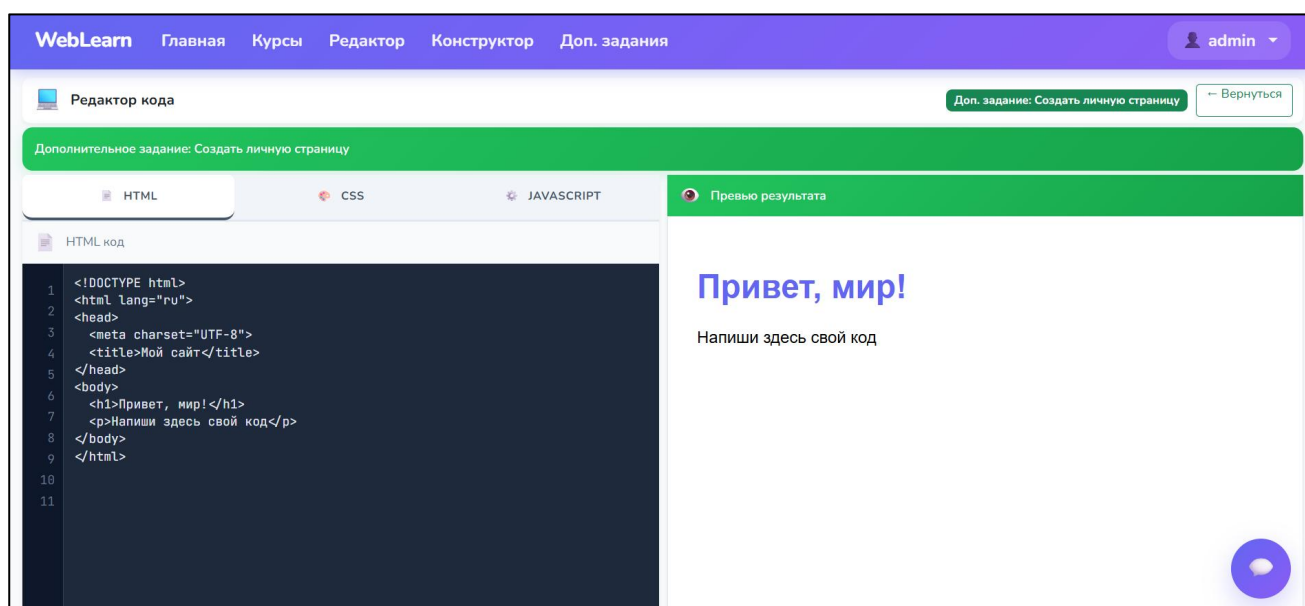
Перед началом выполнения задания необходимо определить общую структуру будущей страницы, продумать расположение основных блоков и выделить ключевые элементы интерфейса. Это помогает сформировать логичную композицию и избежать хаотичного размещения контента. Далее следует подготовить базовую HTML-разметку, соблюдая семантику и корректную вложенность элементов.

Порядок выполнения задания:

1. Определите структуру будущей страницы и необходимые элементы.
2. Подготовьте базовую HTML-разметку.
3. Добавьте CSS-оформление согласно требованиям.
4. При необходимости реализуйте интерактивность на JavaScript.
5. Проведите визуальную и функциональную проверку.
6. Оформите комментарий и отправьте работу на проверку.

Критерии оценивания:

- критерий 1: корректность структуры и семантики HTML-кода;
- критерий 2: чистота, аккуратность и читаемость кода;
- критерий 3: соответствие макету или техническому заданию;
- критерий 4: работоспособность интерактивных элементов;
- критерий 5: аккуратность и завершённость итогового результата.



5.2. Создание интерактивной формы

В каждом учебном сценарии представлена понятная задача, пошаговый алгоритм выполнения и критерии оценки результата. Такой подход помогает структурировать работу, последовательно выполнять задания и контролировать качество итогового решения.

Перед началом разработки необходимо определить структуру будущей формы и список обязательных полей ввода. Это может быть форма регистрации, обратной связи или иная форма для сбора пользовательских данных. Важно заранее продумать логику взаимодействия с пользователем и определить, какие данные будут обрабатываться.

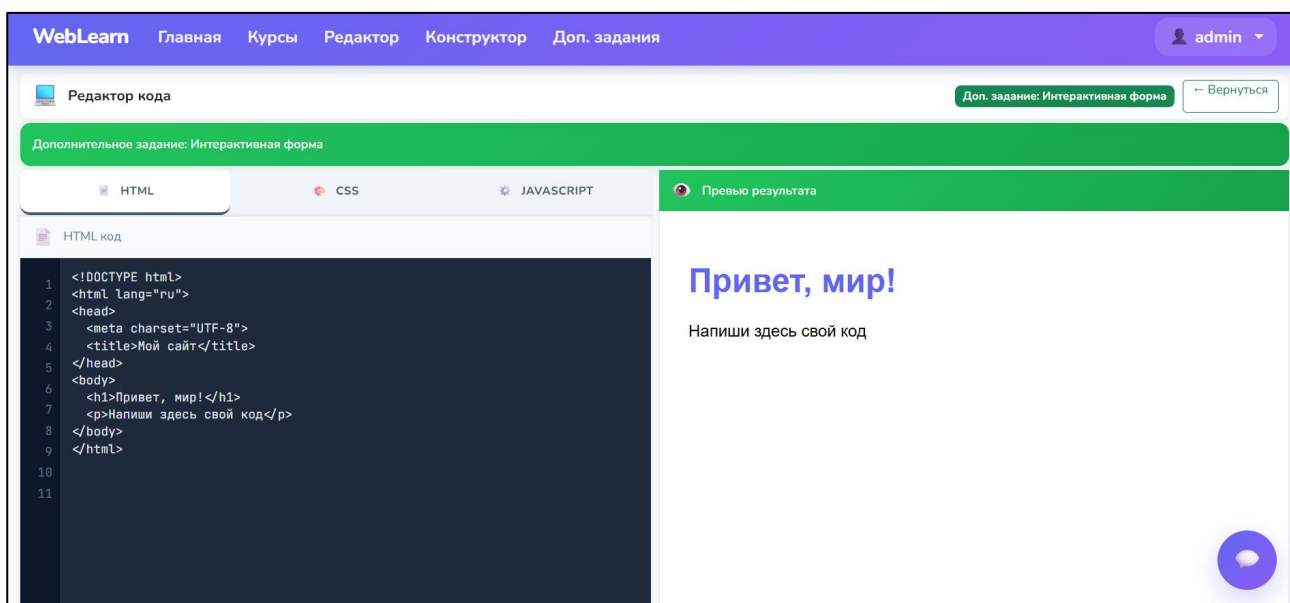
Далее следует подготовить HTML-разметку формы, соблюдая семантику и корректную структуру элементов. После этого добавляется CSS-оформление, которое отвечает за внешний вид полей ввода, кнопок и сообщений об ошибках или успешной отправке. Оформление должно быть понятным, удобным и визуально аккуратным.

Порядок выполнения задания:

1. Определите структуру формы и необходимые поля ввода.
2. Подготовьте HTML-разметку формы регистрации или обратной связи.
3. Добавьте CSS-оформление для полей, кнопок и сообщений.
4. Реализуйте интерактивность и проверку данных с помощью JavaScript.
5. Проведите тестирование корректности работы формы.
6. Оформите комментарий и отправьте работу на проверку.

Критерии оценивания:

- критерий 1: корректность структуры формы и элементов;
- критерий 2: правильная работа валидации данных;
- критерий 3: удобство и читаемость интерфейса;
- критерий 4: использование JavaScript-логики;
- критерий 5: аккуратность и завершённость итогового результата.



5.3. Создание галереи изображений

В каждом учебном сценарии представлена понятная задача, последовательность шагов выполнения и проверка результата. Такой подход позволяет структурировать процесс работы, поэтапно создавать проект и контролировать качество итогового результата.

Перед началом выполнения задания необходимо определить структуру будущей галереи и количество изображений, которые будут в неё включены. Важно заранее продумать расположение элементов и общий визуальный стиль, чтобы интерфейс выглядел логично и удобно для пользователя.

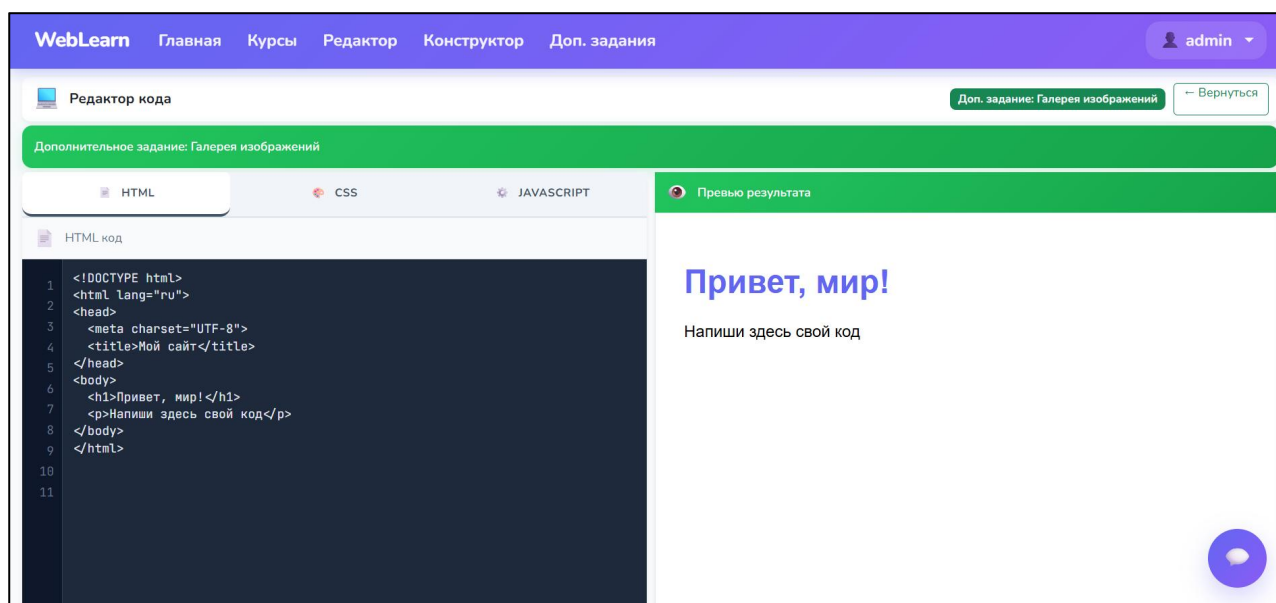
Далее подготавливается HTML-разметка карточек изображений. На этом этапе следует соблюдать семантику и корректную организацию элементов, чтобы каждая карточка была самостоятельным блоком. После этого добавляется CSS-оформление, которое отвечает за внешний вид галереи и сетку расположения элементов. Рекомендуется использовать современные подходы к построению сеток, чтобы обеспечить аккуратное и равномерное размещение изображений.

Порядок выполнения задания:

1. Определите структуру галереи и количество изображений.
2. Подготовьте HTML-разметку карточек изображений.
3. Добавьте CSS-оформление и сетку расположения элементов.
4. При необходимости реализуйте просмотр изображений в полном размере через JavaScript.
5. Проведите визуальную проверку отображения галереи.
6. Оформите комментарий и отправьте работу на проверку.

Критерии оценивания:

- критерий 1: правильность структуры галереи;
- критерий 2: корректное отображение изображений;
- критерий 3: адаптивность и удобство интерфейса;
- критерий 4: наличие интерактивных элементов;
- критерий 5: аккуратность и завершённость оформления страницы.



5.4. Создание калькулятора

В каждом учебном сценарии представлена понятная задача, последовательность шагов выполнения и критерии проверки результата. Такой подход позволяет структурировать процесс разработки, поэтапно создавать функциональный интерфейс и контролировать качество итоговой работы.

Перед началом выполнения задания необходимо определить основные функции калькулятора и набор кнопок, которые будут использоваться для выполнения математических операций. Важно заранее продумать логику работы интерфейса, чтобы обеспечить удобство и понятность для пользователя.

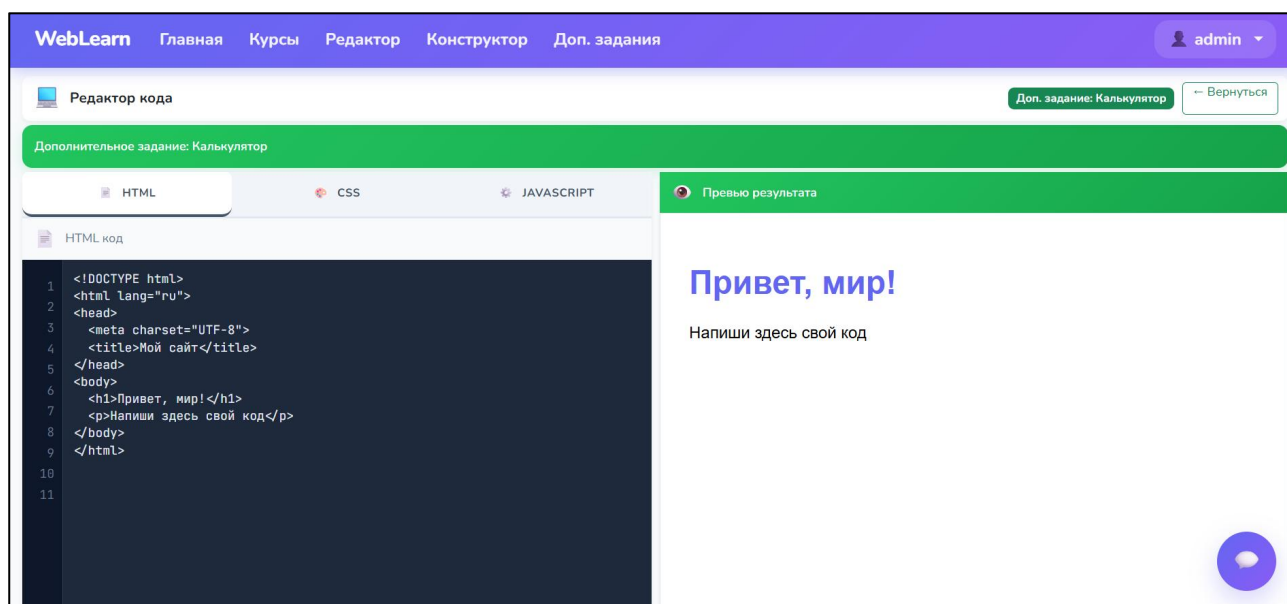
Далее подготавливается HTML-разметка интерфейса калькулятора. На этом этапе создаётся структура кнопок, экрана вывода и основных элементов управления. Затем добавляется CSS-оформление, которое отвечает за внешний вид кнопок, расположение элементов и оформление области отображения результата. Интерфейс должен быть аккуратным, читаемым и удобным для взаимодействия.

Порядок выполнения задания:

1. Определите функции и кнопки будущего калькулятора.
2. Подготовьте HTML-разметку интерфейса калькулятора.
3. Добавьте CSS-оформление кнопок и экрана вывода.
4. Реализуйте вычисления и обработку действий с помощью JavaScript.
5. Проведите проверку корректности математических операций.
6. Оформите комментарий и отправьте работу на проверку.

Критерии оценивания:

- критерий 1: правильность структуры интерфейса;
- критерий 2: корректная работа вычислений;
- критерий 3: чистота и читаемость JavaScript-кода;
- критерий 4: удобство использования калькулятора;
- критерий 5: аккуратность и завершённость итогового результата.



5.5. Создание игры «Угадай число»

В каждом учебном сценарии представлена понятная задача, последовательность шагов выполнения и критерии проверки результата. Такой подход позволяет поэтапно разрабатывать проект, контролировать качество кода и формировать практические навыки работы с HTML, CSS и JavaScript.

Перед началом работы необходимо определить правила и основную логику игры «Угадай число». Следует заранее продумать диапазон чисел, количество попыток и условия победы или проигрыша. Это помогает правильно выстроить структуру будущего приложения и упростить реализацию логики.

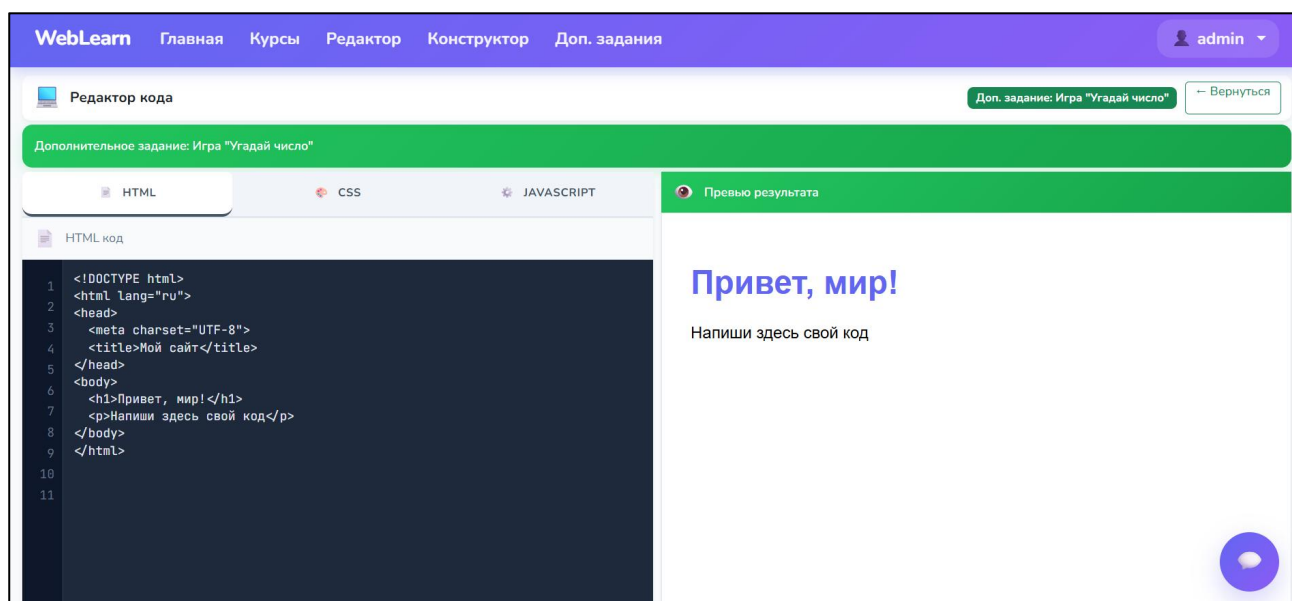
Далее подготавливается HTML-разметка игрового интерфейса. На этом этапе создаются элементы для ввода числа, отображения сообщений пользователю и управления игрой. После этого добавляется CSS-оформление, которое отвечает за внешний вид элементов, их расположение и удобство восприятия интерфейса.

Порядок выполнения задания:

1. Определите правила и логику игры.
2. Подготовьте HTML-разметку игрового интерфейса.
3. Добавьте CSS-оформление элементов игры.
4. Реализуйте игровую логику и обработку действий пользователя с помощью JavaScript.
5. Проведите тестирование корректности работы игры.
6. Оформите комментарий и отправьте работу на проверку.

Критерии оценивания:

- критерий 1: корректность игровой логики;
- критерий 2: правильная работа JavaScript-сценариев;
- критерий 3: удобство взаимодействия пользователя;
- критерий 4: соответствие условиям задания;
- критерий 5: аккуратность и завершённость итогового результата.



5.6. Создание интернет-магазина

В каждом учебном сценарии представлена понятная задача, последовательность шагов выполнения и критерии проверки результата. Такой подход позволяет системно подойти к разработке проекта, постепенно формируя функциональный и структурированный веб-интерфейс.

Перед началом работы необходимо определить общую структуру интернет-магазина и перечень ключевых элементов страницы. Важно продумать расположение каталога товаров, меню навигации, корзины покупок и дополнительных блоков, чтобы обеспечить удобство взаимодействия пользователя с интерфейсом.

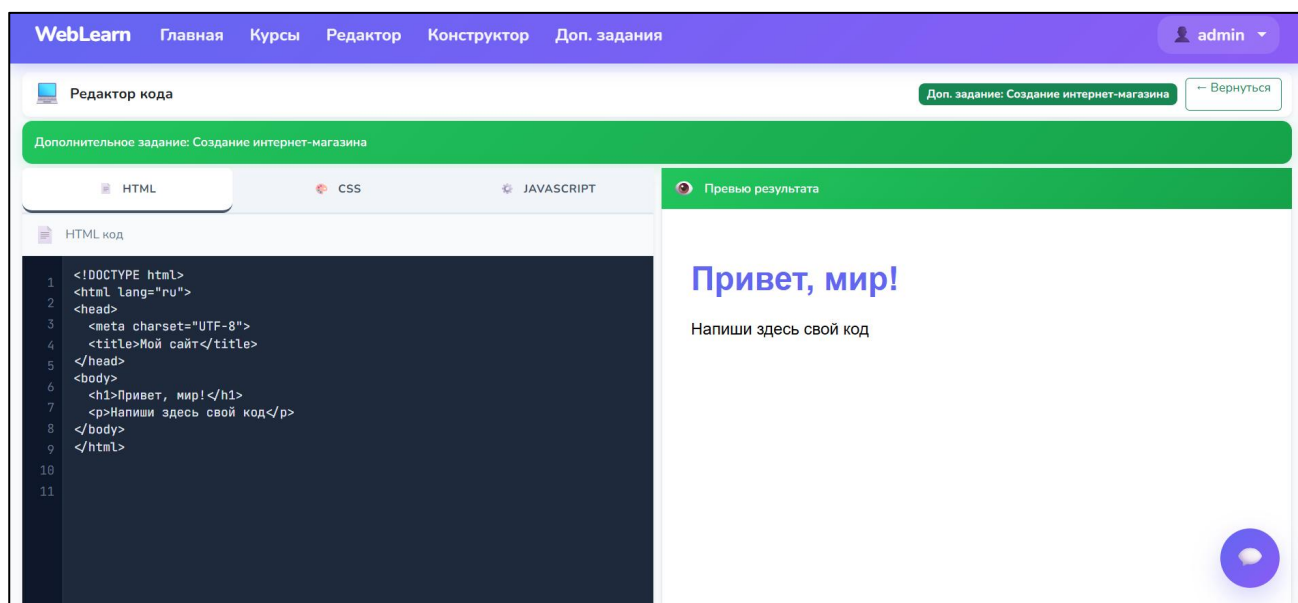
Далее подготавливается HTML-разметка карточек товаров, навигационного меню и корзины. На этом этапе особое внимание уделяется семантике и логичной организации элементов страницы.

Порядок выполнения задания:

1. Определите структуру интернет-магазина и необходимые элементы страницы.
2. Подготовьте HTML-разметку карточек товаров, меню и корзины.
3. Добавьте CSS-оформление для каталога, кнопок и изображений товаров.
4. Реализуйте интерактивность кнопок и элементов корзины с помощью JavaScript.
5. Проведите визуальную и функциональную проверку работы страницы.
6. Оформите комментарий и отправьте работу на проверку.

Критерии оценивания:

- критерий 1: корректность структуры и семантики HTML;
- критерий 2: чистота и читаемость кода;
- критерий 3: соответствие макету или техническому заданию;
- критерий 4: работоспособность интерактивных элементов;
- критерий 5: аккуратность и завершённость итогового результата.



5.7. Создание мини-социальной сети

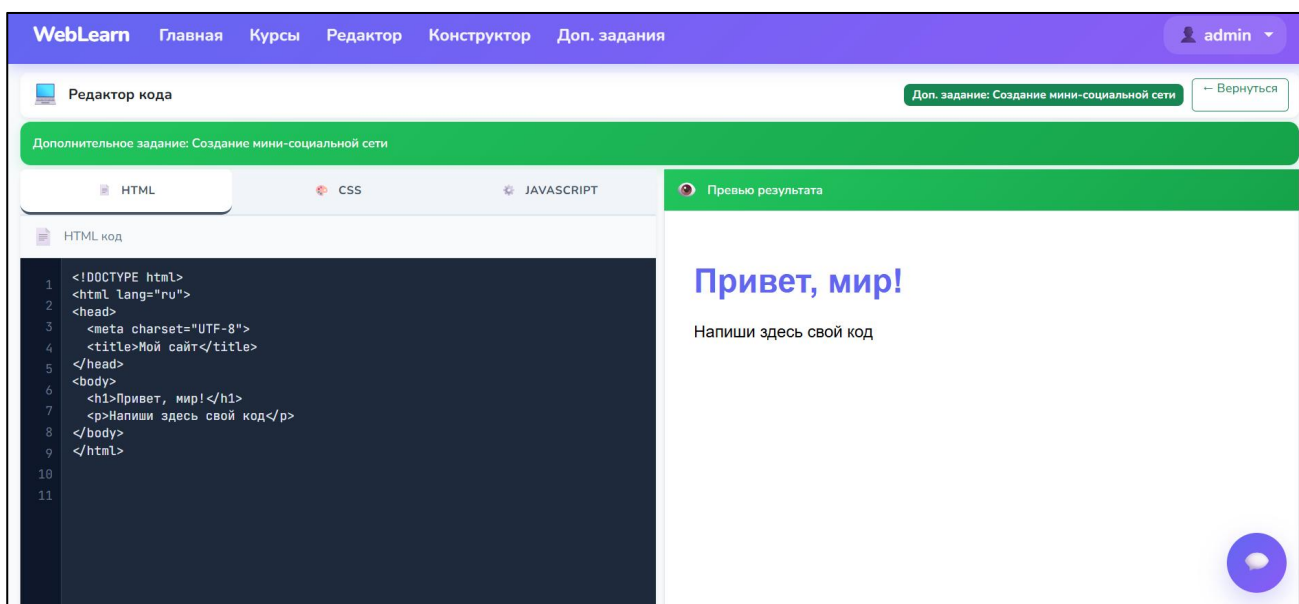
В каждом учебном сценарии представлена понятная задача, последовательность шагов выполнения и критерии проверки результата. Такой подход позволяет поэтапно разрабатывать проект, формировать структурированный код и закреплять навыки работы с HTML, CSS и JavaScript. Перед началом работы необходимо определить структуру страницы профиля пользователя и блоков с публикациями. Важно заранее продумать, какие элементы будут включены в интерфейс: информация профиля, лента постов, комментарии и элементы взаимодействия. Это помогает создать логичную и удобную архитектуру страницы.

Далее подготавливается HTML-разметка профиля, публикаций и комментариев. На этом этапе важно соблюдать семантику и корректную вложенность элементов, чтобы структура документа была понятной и легко поддерживаемой. Порядок выполнения задания:

1. Определите структуру страницы профиля и публикаций.
2. Подготовьте HTML-разметку профиля, постов и комментариев.
3. Добавьте CSS-оформление интерфейса социальной сети.
4. Реализуйте интерактивность публикаций и комментариев с помощью JavaScript.
5. Проведите визуальную и функциональную проверку страницы.
6. Оформите комментарий и отправьте работу на проверку.

Критерии оценивания:

- критерий 1: корректность структуры и семантики HTML;
- критерий 2: чистота и читаемость кода;
- критерий 3: соответствие макету или техническому заданию;
- критерий 4: работоспособность интерактивных элементов;
- критерий 5: аккуратность и завершённость итогового результата.



5.8. Создание To-Do List

В каждом учебном сценарии представлена понятная задача, последовательность шагов выполнения и критерии проверки результата. Такой подход позволяет поэтапно создавать проект, структурировать код и закреплять навыки работы с HTML, CSS и JavaScript.

Перед началом работы необходимо определить структуру списка задач и основные функции будущего приложения. Важно продумать, какие действия сможет выполнять пользователь: добавление новых задач, удаление выполненных и изменение существующих элементов. Это помогает заранее сформировать логику работы приложения.

Далее подготавливается HTML-разметка списка задач и элементов управления. На этом этапе создаются поле ввода, кнопка добавления и контейнер для отображения задач. После этого добавляется CSS-оформление, которое отвечает за внешний вид задач, кнопок и формы ввода. Интерфейс должен быть простым, удобным и понятным для пользователя.

Порядок выполнения задания:

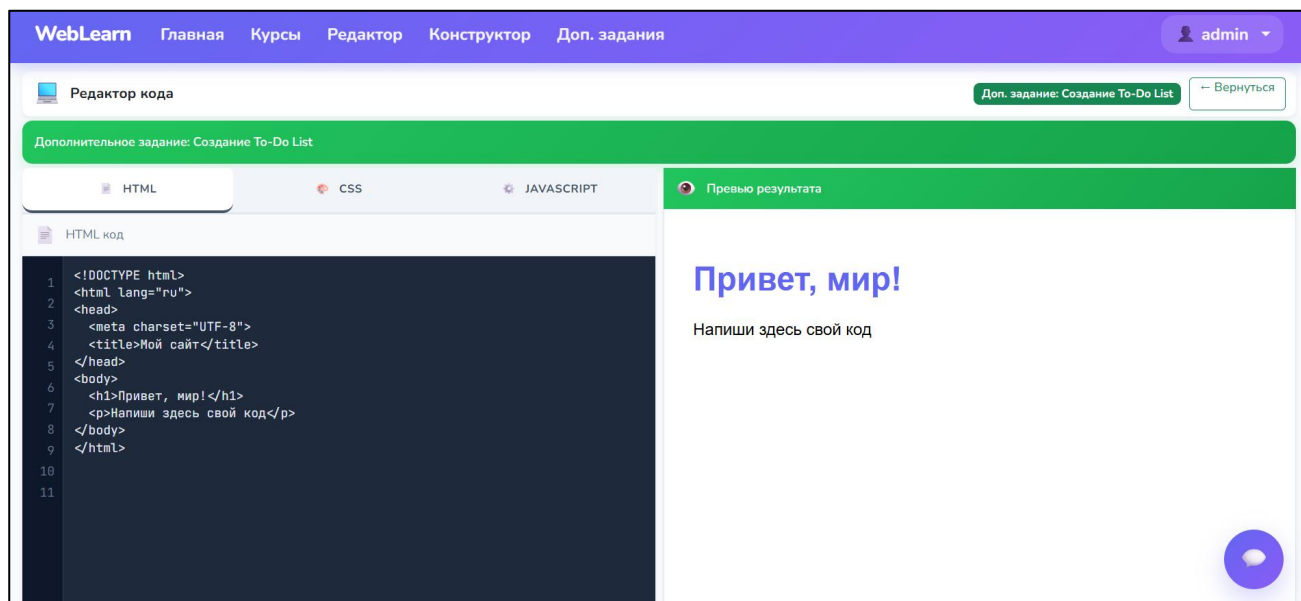
1. Определите структуру списка задач и необходимые функции.
2. Подготовьте HTML-разметку списка и элементов управления.
3. Добавьте CSS-оформление для задач, кнопок и формы ввода.
4. Реализуйте добавление, удаление и изменение задач с помощью

JavaScript.

5. Проведите проверку корректности работы списка задач.
6. Оформите комментарий и отправьте работу на проверку.

Критерии оценивания:

- критерий 1: корректность структуры и семантики HTML;
- критерий 2: чистота и читаемость кода;
- критерий 3: соответствие макету или техническому заданию;
- критерий 4: работоспособность интерактивных элементов;
- критерий 5: аккуратность и завершённость итогового результата.



5.9. Создание онлайн-теста

В каждом учебном сценарии представлена понятная задача, последовательность шагов выполнения и проверка результата. Такой подход позволяет структурировать процесс разработки, поэтапно создавать функциональный проект и закреплять навыки работы с HTML, CSS и JavaScript. Перед началом работы необходимо определить структуру теста и количество вопросов. Важно заранее продумать логику прохождения тестирования, формат ответов и способ подсчёта результатов. Это позволяет создать понятный и удобный интерфейс для пользователя.

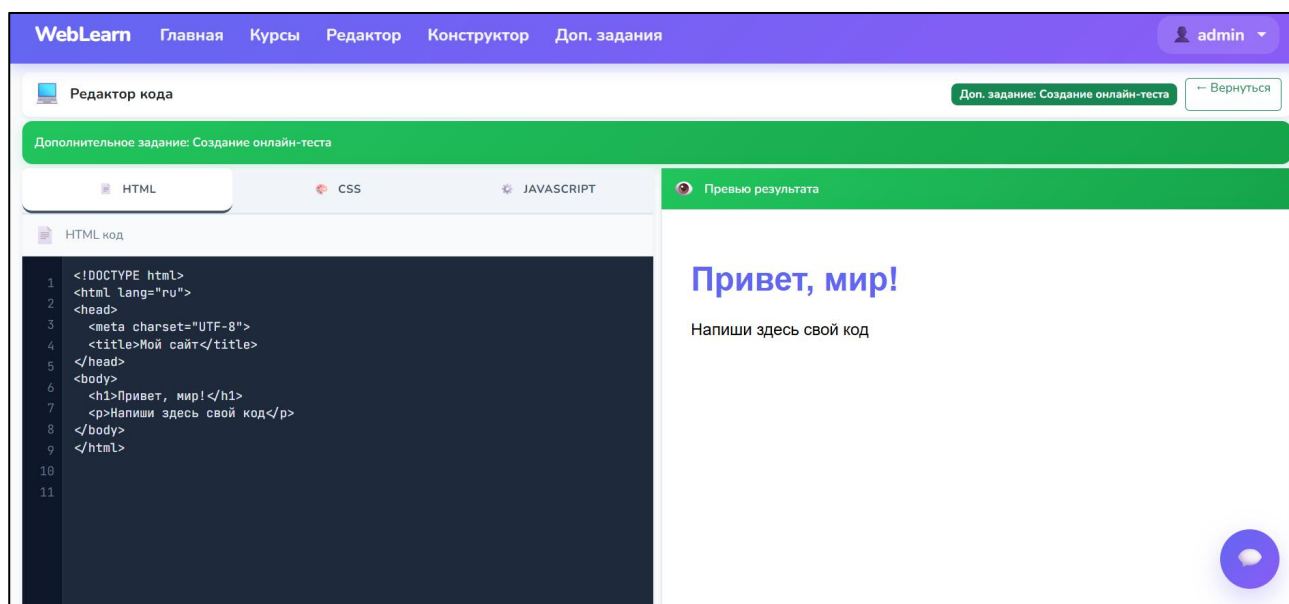
Далее подготавливается HTML-разметка вопросов и вариантов ответов. На этом этапе создаются элементы для отображения вопросов, вариантов выбора и кнопки управления тестом. После этого добавляется CSS-оформление, которое отвечает за внешний вид интерфейса тестирования, включая расположение элементов, читаемость текста и общее визуальное оформление.

Порядок выполнения задания:

1. Определите структуру теста и количество вопросов.
2. Подготовьте HTML-разметку вопросов и вариантов ответов.
3. Добавьте CSS-оформление интерфейса тестирования.
4. Реализуйте подсчёт результатов и проверку ответов с помощью JavaScript.
5. Проведите визуальную и функциональную проверку работы теста.
6. Оформите комментарий и отправьте работу на проверку.

Критерии оценивания:

- критерий 1: корректность структуры и семантики HTML;
- критерий 2: чистота и читаемость кода;
- критерий 3: соответствие макету или техническому заданию;
- критерий 4: работоспособность интерактивных элементов;
- критерий 5: аккуратность и завершённость итогового результата.



6. Система контроля и оценивания

Система контроля и оценивания на среде WebLearn предназначена для проверки уровня усвоения учебного материала, оценки практических навыков обучающихся и своевременного выявления ошибок в процессе обучения. Оценивание строится на понятных и прозрачных критериях, что позволяет пользователям понимать требования к выполнению заданий и отслеживать собственный прогресс.

Основное внимание при проверке работ уделяется не только правильности выполнения задания, но и качеству программного кода, соблюдению структуры проекта, аккуратности оформления и самостоятельности выполнения работы. Такой подход помогает формировать у обучающихся навыки грамотной организации кода и культуры веб-разработки.

Для более объективной оценки результатов обучения используется система уровней освоения материала.

Базовый уровень. Обучающийся выполнил основную часть задания, однако в работе присутствуют отдельные недочёты или ошибки. Код может содержать незначительные проблемы в структуре, оформлении или логике работы программы. Задание в целом соответствует требованиям, но требует дополнительной доработки и исправления замечаний.

Достаточный уровень. Задание выполнено корректно в соответствии с основными требованиями. Код работает стабильно, структура проекта организована правильно, а основные элементы интерфейса реализованы без существенных ошибок. Обучающийся демонстрирует понимание изученного материала и способность самостоятельно выполнять практические задания.

Повышенный уровень. Работа выполнена аккуратно и качественно. Код отличается хорошей читаемостью, логичной структурой и правильным использованием технологий HTML, CSS и JavaScript. Обучающийся проявляет высокую степень самостоятельности, использует дополнительные возможности оформления и оптимизации проекта, а также соблюдает рекомендации по организации кода.

Высокий уровень. Задание выполнено на высоком профессиональном уровне. Проект имеет продуманную структуру, оптимизированный код и качественно реализованный интерфейс. Пользователь демонстрирует уверенное владение изученными технологиями, самостоятельно применяет дополнительные решения и улучшения, а также способен эффективно исправлять ошибки и дорабатывать проект после проверки.

Для повышения эффективности обучения рекомендуется проводить короткие проверки после завершения каждого учебного модуля, практического задания или мини-проекта. Регулярный контроль позволяет своевременно обнаруживать ошибки, корректировать понимание материала и постепенно повышать уровень подготовки обучающихся. Кроме того, систематическая проверка способствует формированию устойчивых практических навыков веб-программирования и повышает качество итоговых работ.

7. Типичные ошибки и способы их устранения

Ошибка №1

Проблема: HTML-теги отображаются как обычный текст на странице.

Решение: Проверьте правильность написания тегов и наличие угловых скобок `<>`.

Ошибка №2

Проблема: Изображение не отображается на странице.

Решение: Проверьте путь к изображению в атрибуте `src` и наличие файла в папке проекта.

Ошибка №3

Проблема: CSS-стили не применяются к странице.

Решение: Проверьте правильность подключения CSS-файла через тег `<link>` и путь к файлу.

Ошибка №4

Проблема: Элемент имеет неправильный цвет или размер.

Решение: Проверьте CSS-селектор и правильность написания свойств.

Ошибка №5

Проблема: Не работает класс (`class`) или идентификатор (`id`).

Решение: Проверьте совпадение имени класса или идентификатора в HTML и CSS.

Ошибка №6

Проблема: Сайт отображается некорректно на разных устройствах.

Решение: Используйте адаптивную верстку и проверьте наличие метатега `viewport`.

Ошибка №7

Проблема: JavaScript-код не выполняется.

Решение: Проверьте подключение JS-файла и наличие ошибок в консоли браузера.

Ошибка №8

Проблема: Кнопка не реагирует на нажатие.

Решение: Проверьте правильность обработки события `onclick` и название функции.

Ошибка №9

Проблема: Переменная выводит ошибку `"undefined"`.

Решение: Убедитесь, что переменная была объявлена и инициализирована до её использования.

Ошибка №10

Проблема: Условие работает неправильно.

Решение: Проверьте операторы сравнения (`===`, `==`, `!=`) и логические выражения.

Ошибка №11

Проблема: Цикл выполняется бесконечно.

Решение: Проверьте условие завершения цикла и корректное изменение счётчика.

Ошибка №12

Проблема: Функция не вызывается.

Решение: Проверьте имя функции и место её вызова в коде.

Ошибка №13

Проблема: Не изменяется текст или стиль элемента через JavaScript.

Решение: Проверьте правильность обращения к DOM-элементу и существование указанного id.

Ошибка №14

Проблема: Форма отправляется с пустыми или некорректными данными.

Решение: Добавьте валидацию полей и проверьте работу функции проверки данных.

Ошибка №15

Проблема: Элемент перекрывает другой блок на странице.

Решение: Проверьте position, z-index и структуру вложенности элементов.

Ошибка №16

Проблема: Flexbox-элементы располагаются неправильно.

Решение: Проверьте flex-direction, justify-content и align-items.

Ошибка №17

Проблема: Grid-сетка ломается или не выстраивается.

Решение: Проверьте grid-template-columns, grid-template-rows и размеры контейнера.

Ошибка №18

Проблема: Шрифт не отображается на странице.

Решение: Проверьте подключение шрифта и корректность @font-face или Google Fonts.

Ошибка №19

Проблема: Ссылки не работают или ведут не туда.

Решение: Проверьте атрибут href и правильность относительных/абсолютных путей.

Ошибка №20

Проблема: Страница загружается слишком медленно.

Решение: Оптимизируйте изображения, уменьшите количество подключаемых ресурсов и проверьте размер файлов.

Ошибка №21

Проблема: Анимации работают с задержкой или дергаются.

Решение: Проверьте свойства transition, animation и избегайте перегрузки эффектами.

Ошибка №22

Проблема: Скрипт выполняется до загрузки HTML-элементов.

Решение: Переместите <script> в конец <body> или используйте DOMContentLoaded.

8. Организация самостоятельной работы обучающихся

Самостоятельная работа обучающихся является важной частью процесса изучения веб-программирования и направлена на закрепление теоретических знаний, развитие практических навыков и формирование способности самостоятельно решать учебные и прикладные задачи. Работа со средой WebLearn позволяет пользователям изучать материал в удобном темпе, выполнять практические задания и постепенно совершенствовать навыки разработки веб-приложений.

Для эффективной организации самостоятельной работы рекомендуется заранее планировать учебную деятельность, распределять нагрузку и регулярно отслеживать результаты обучения. Систематический подход помогает обучающимся лучше усваивать материал, избегать перегрузки и своевременно устранять возникающие ошибки. Рекомендуется придерживаться следующего порядка организации самостоятельной работы:

1. Составление недельного учебного плана. Перед началом учебного периода обучающемуся рекомендуется определить цели на неделю и составить краткий план работы. В плане следует указать темы для изучения, количество практических заданий и предполагаемое время выполнения работы. Приоритет рекомендуется отдавать тем разделам, которые вызывают наибольшие затруднения или требуют дополнительной практики.

2. Определение минимального объёма практических заданий. Для поддержания стабильного прогресса необходимо определить минимальное количество практических заданий, которое должно быть выполнено за определённый период. Регулярное выполнение даже небольших заданий способствует более эффективному закреплению навыков работы с HTML, CSS и JavaScript.

3. Проведение самопроверки после выполнения заданий. После завершения каждого задания рекомендуется проводить самостоятельную проверку результата по заранее подготовленному чек-листу. Во время проверки следует обратить внимание на:

- корректность работы кода;
- наличие ошибок в структуре HTML-документа;
- правильность подключения CSS и JavaScript;
- аккуратность оформления интерфейса;
- читаемость и логичность программного кода;
- соответствие задания поставленным требованиям.

4. Развитие навыков самостоятельного поиска информации. В процессе обучения веб-технологиям важно уметь находить и анализировать дополнительные источники знаний. Это могут быть официальная документация, образовательные платформы, профессиональные блоги и форумы разработчиков. Самостоятельный поиск информации позволяет быстрее разбираться в новых инструментах, понимать актуальные подходы в разработке и расширять кругозор. Особенно важно учиться оценивать достоверность источников, сравнивать разные решения одной задачи и выбирать наиболее эффективные из них.

5. Практика закрепления теоретических знаний. Любая изученная тема должна сопровождаться практическими заданиями. Например, после изучения HTML и CSS полезно создавать небольшие веб-страницы, а после освоения JavaScript — реализовывать простые интерактивные элементы. Регулярная практика помогает закрепить материал, развивает логическое мышление и формирует уверенные навыки программирования. Чем больше разнообразных задач выполняет обучающийся, тем легче ему адаптироваться к реальным проектам.
6. Работа над ошибками в процессе обучения. Ошибки являются естественной частью учебного процесса, особенно в программировании. Важно не только исправлять их, но и понимать причину возникновения. Анализ ошибок помогает выявить пробелы в знаниях и устранить их. Полезно вести отдельные заметки с типичными проблемами и способами их решения, чтобы в дальнейшем быстрее справляться с подобными ситуациями.
7. Использование проектов для закрепления навыков. Одним из наиболее эффективных способов обучения является выполнение мини-проектов. Это могут быть личные сайты, простые веб-приложения или учебные задания с расширенными функциями. Проектная работа позволяет объединить знания из разных областей и учит планировать структуру будущего продукта. Кроме того, она развивает навыки самостоятельного принятия решений и поиска оптимальных решений задач.
8. Формирование устойчивых учебных привычек. Регулярность занятий играет ключевую роль в успешном освоении веб-технологий. Рекомендуется выделять фиксированное время для обучения, избегать длительных перерывов и постепенно увеличивать сложность задач. Постоянная практика помогает сформировать дисциплину и делает процесс обучения более системным и эффективным.

9. Заключение

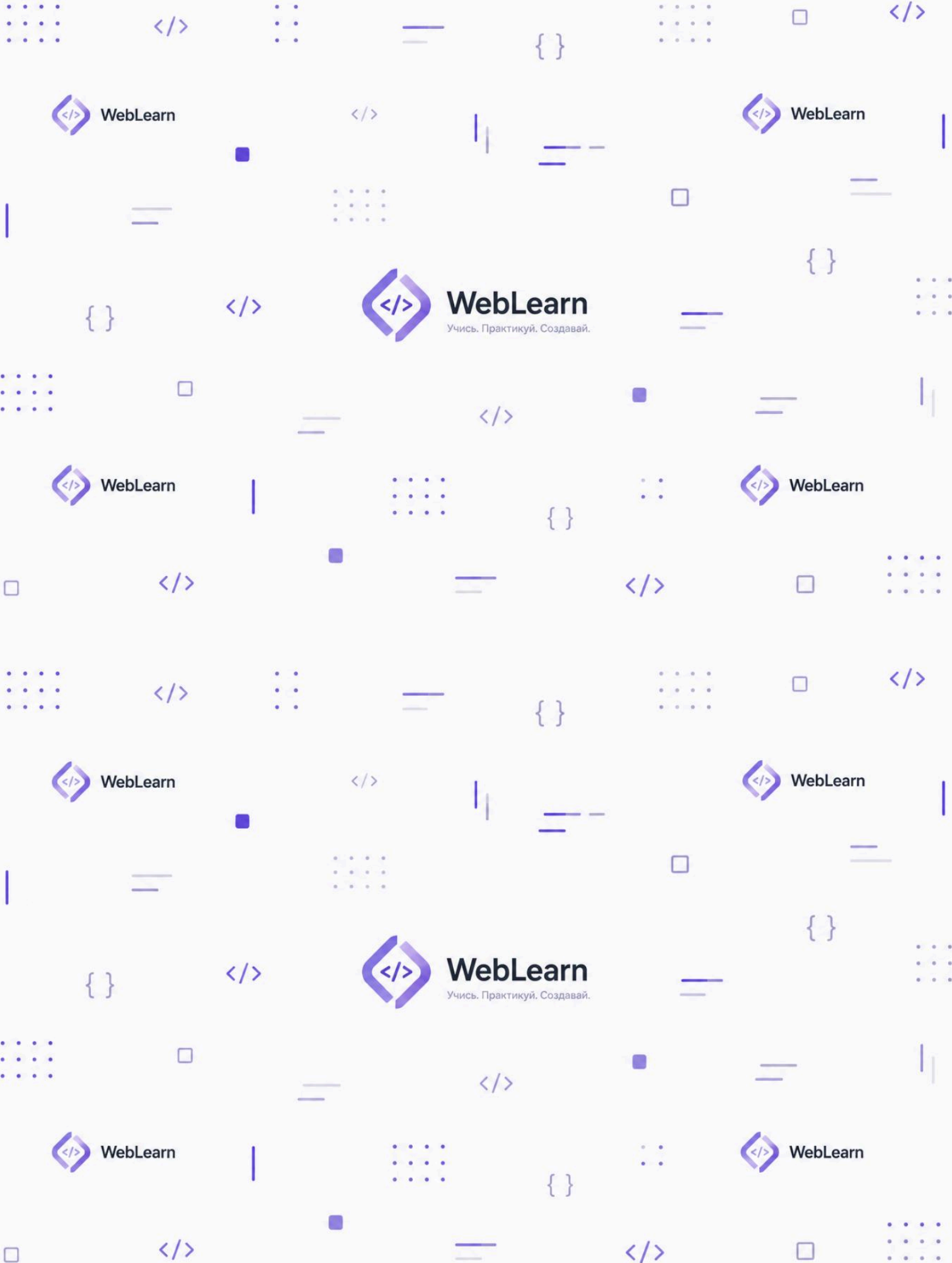
Визуальная среда WebLearn представляет собой современную учебную среду, предназначенную для поэтапного и практико-ориентированного изучения основ веб-разработки. Использование визуальной среды позволяет обучающимся последовательно осваивать технологии HTML, CSS и JavaScript, сочетая изучение теоретического материала с выполнением практических заданий и созданием собственных веб-проектов. Объединение теории, практики, встроенного редактора кода и визуального конструктора в одной системе делает процесс обучения более удобным, доступным и эффективным.

В ходе работы со средой пользователи получают возможность не только изучать основы веб-программирования, но и развивать практические навыки создания веб-страниц, оформления интерфейсов и реализации интерактивных элементов. Постепенное усложнение заданий способствует формированию устойчивых знаний, развитию логического мышления, внимательности, самостоятельности и способности применять полученные навыки при решении практических задач.

Особую роль в обучении играет система практических и дополнительных заданий, позволяющая обучающимся выполнять проекты различного уровня сложности — от простых HTML-страниц и форм обратной связи до интерактивных игр, тестов, калькуляторов и элементов интернет-магазинов. Такой подход помогает постепенно повышать уровень подготовки, расширять опыт программирования и формировать навыки проектной деятельности.

Важным преимуществом среды является возможность получения обратной связи от преподавателя. Проверка выполненных работ, комментарии и рекомендации помогают пользователям своевременно выявлять ошибки, улучшать качество программного кода и учиться соблюдать основные принципы веб-разработки. Регулярная доработка проектов способствует более глубокому пониманию изучаемого материала и формированию навыков самостоятельного анализа своей работы.

Руководство по использованию среды WebLearn может применяться как в образовательном процессе на уроках информатики и программирования, так и для самостоятельного обучения в домашних условиях. Пошаговые инструкции, понятная структура материалов, практическая направленность заданий и визуальная поддержка делают среду удобным инструментом для изучения современных веб-технологий и формирования базовых компетенций в области веб-разработки.





WebLearn

Учись. Практикуй. Создавай.



WebLearn — современная визуальная учебная среда для изучения веб-разработки.



Среда разработана для наглядного и практико-ориентированного обучения HTML, CSS и JavaScript. Благодаря одновременному отображению кода и результата обучающиеся могут сразу видеть изменения и лучше понимать принципы создания веб-страниц.



Визуальная среда включает:



Теоретические материалы по основам веб-разработки



Интерактивный редактор кода с мгновенным предпросмотром



Практические задания различного уровня сложности



Автоматическую проверку выполненных работ



Систему отслеживания учебного прогресса



Конструктор веб-страниц для визуального освоения структуры сайта



Использование **WebLearn** способствует развитию алгоритмического мышления, формированию практических навыков программирования и повышению мотивации обучающихся к изучению современных веб-технологий.